POLITECNICO
MILANO 1863

COMPUTER SCIENCE AND ENGINEERING
SOFTWARE ENGINEERING II
2024 - 2025

DD

Design Document

Students & Companies

Contents

1. **Introduction**
    - 1.1 Purpose
    - 1.2 Scope
    - 1.3 Definitions, Acronyms, Abbreviations
    - 1.4 Revision History
    - 1.5 Document Structure

2. Architectural Design

- 2.1 Overview: High level components and their interaction
- 2.2 Component view
- 2.3 Deployment view
- 2.4 Runtime view
- 2.5 Component interfaces
- 2.6 Selected architectural styles and patterns
- 2.7 Other design decisions

3. User Interface Design

4. Requirements Traceability

5. Implementation , Integration & Testing

- 5.1 Overview
- 5.2 Implementation Plan
    - 5.2.1 Features identification
- 5.3 Component Integration and Testing
- 5.4 System Testing
- 5.5 Additional Specifications on Testing

6. Effort Spent

7. References

1. Introduction

1.1 Purpose

In the modern industry, rapid advancements in technology and evolving market demands have created a dynamic and competitive environment where skilled talent is paramount. Companies are constantly seeking innovative minds to bring fresh perspectives to their projects, while students aim to gain real-world experience to complement their academic knowledge. Bridging the gap between these two groups has become increasingly important to address the challenges of workforce readiness and talent acquisition. S&C is a well-designed platform that connects companies and students that provides a structured pathway for meaningful internships, enabling students to gain hands-on experience while contributing to impactful projects. Such initiatives not only empower students to build their professional profiles but also help companies cultivate future-ready talent, creating a mutually beneficial ecosystem.

1.2 Scope

The scope of the system is to facilitate seamless interaction and collaboration between students, companies, and universities by acting as a mediator throughout the internship process. The platform enables matchmaking by allowing students to upload their CVs, which serve as the foundation for recommending suitable internship opportunities, and by requiring companies to provide detailed descriptions of the projects they wish to offer. All users interact with the platform through dedicated accounts, with students having personal accounts and companies and universities representing their organizations. To ensure reliability, companies and universities undergo a verification process conducted by the S&C administrators. The system supports direct communication between students and companies at every stage, from negotiating internship details to finalizing contracts, while also enabling universities to oversee their students' internships and address any complaints that arise. Additionally, it offers monitoring tools for tracking ongoing internships and facilitates decision-making regarding their continuation or premature termination. Once a contract is finalized and accepted, the system officially records the collaboration, marking the start of the internship. By integrating these features, the platform establishes a structured and efficient ecosystem for fostering meaningful collaborations among all stakeholders.

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

- **Internship**: Temporary position held by a student for a specified period, allowing them to work under the supervision of professionals in a company. Internships can be paid or unpaid and offer both tangible and intangible benefits, such as training and mentorship.
- **Recommender system**: A mechanism that alerts students when a new project of interest becomes available and notifies companies when a new CV matching their requirements is published. It incorporates statistical data derived from feedback provided by students and companies during internships. This data is analyzed and utilized to enhance the system's effectiveness in making recommendations.
- **Matchmaking process**: The process that begins when a student and a company connect and concludes when the final contract is signed or one of the parties decides to discontinue. It includes communication through the platform, the interview process, and the proposal phase.
- **Handshake**: The first connection request carried out by one of the two parties. If accepted it marks the beginning of the matchmaking process.
- **Explore:** The section of the platform that offers support to both students and companies in finding valuable opportunities, and that provides recommendations and notifications about positions and CVs uploaded on the system.
- **Legal Documentation:** the collection of official documents that describes an organization's legal status and ensures its reliability.
- **Organization**: refers to both companies and universities.

1.3.2 Acronyms

- **S&C**: Students and Companies.
- **CV**: Curriculum Vitae.

1.3.3 Abbreviations

- **IdP**: Identity Provider
- **SP**: Service Provider

1.4 Revision History

- Version 1.0: 06/01/2025

1.5 Reference Documents

The assignment for this document and all the information included refer to the following documentation:

- The specification for the 2024/25 Requirement Engineering and Design Project for the Software Engineering II course.
- The slides on the webeep page of the Software Engineering II course.
- The Requirement Analysis and Specification Document.

1.6 Document Structure

1. **Introduction**: it aims to give a brief description of the project. In particular it's focused on the reasons and the goals that are going to be achieved with its development, as previously explained in the RASD;
2. **Architectural Design**: : This part of the document describes at different levels of granularity how the architecture is designed;
3. **User Interface Design**: This section shows the wire frame that constitutes the platform UI;
4. **Requirements Traceability**: in this section it is explained how the goals defined in the previous document are satisfied by the interaction between the requirements and the component described in this document.
5. **Implementation, Integration and Test Plan**: this section provides a descrip- tion of the implementation integration and testing details, useful for the developers and producer of the platform;
6. **Effort Spent**: shows the effort spent to write this document
7. **Bibliography**: it contains the references to any documents and software used to write this paper.

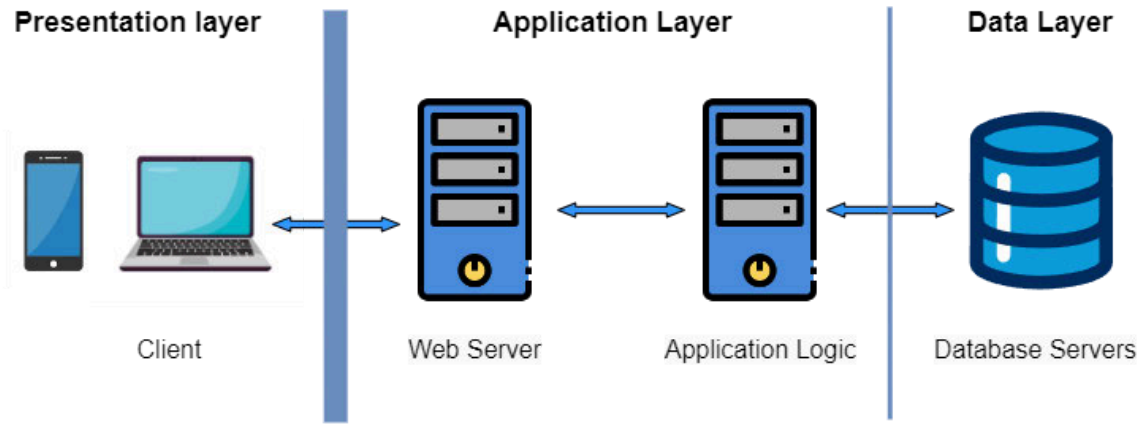
2. Architectural Design

2.1 Overview

This section consists of a high level description of the architecture of the system, of the single components and the interaction between the architectural layers. It also includes the modeling choices that have been taken to fulfill the functional and nonfunctional requirements defined in the RASD document.

2.1.1 System View

The optimal solution for the architectural design of the S&C platform is to implement a 4-tiered architecture. These four tiers consist of one for the presentation layer, two for the application layer, and one for the data layer. The presentation layer will reside on the client, and will be responsible for displaying the user interface through a web browser. The application layer, located at server side, will be divided into two tiers: one including the web server, the API gateway and a proxy for API calls, the other hosting the main application services essential for functionalities offered by the system.
The data tier will house the databases and the DBMS necessary for their operation. Information flow within the system moves from the front end to the back end, starting from the presentation layer where users interact with the system, through the web server, the API gateway and the components of the application layer that interpret requests and interact with the databases, and finally to the data layer. Once the data is retrieved and processed by the relevant services, it is prepared, formatted, and conveyed back to the presentation layer for display to the user.
As the main architectural style for the application tier, the system uses microservices, which are a flexible solution useful in the case of systems that scale rapidly. The microservices approach consists in the implementation and integration of many small independent services that communicate through REST API calls. Both the application and data layers must be replicated over different machines and need to be redundant in order to grant availability and fault tolerance.



2.1.2 Detailed View

Presentation Tier

The user interacts with the system from their device only through the web browser, which represents the presentation layer. The browser is responsible for making HTTP requests to a web server of the platform and to receive the response with the formatted data that is then displayed to the user.

Application Tier - Front End

This application tier comprises all those elements of the system that constitute the point of contact between the network and the core parts of the system. Some of the key components it hosts include the web server, which is responsible for receiving user requests, the API gateway, which handles functions such as request routing, load balancing, caching, and the proxy for API calls. This last component is used by the system as an access point to external APIs it needs to utilize, such as the authentication features of the identity provider, Google Maps APIs, and email services. When a new user request is received by the web server, it is passed to the API gateway which performs a series of operations: ensures that the request originates from an authenticated and authorized user with access to the information contained within the system; then it attempts to optimize the response using mechanisms such as caching or aggregating multiple requests into one. Finally, it handles the routing of requests by acting as a load balancer, directing them to an available application server that hosts the service required. This component, along with the entire tier, also provides an additional layer of security by mitigating potential threats such as SQL Injections or DoS attacks before they can target the primary application servers, and allows for the implementation of another firewall that separates it from the most vulnerable components of the system.

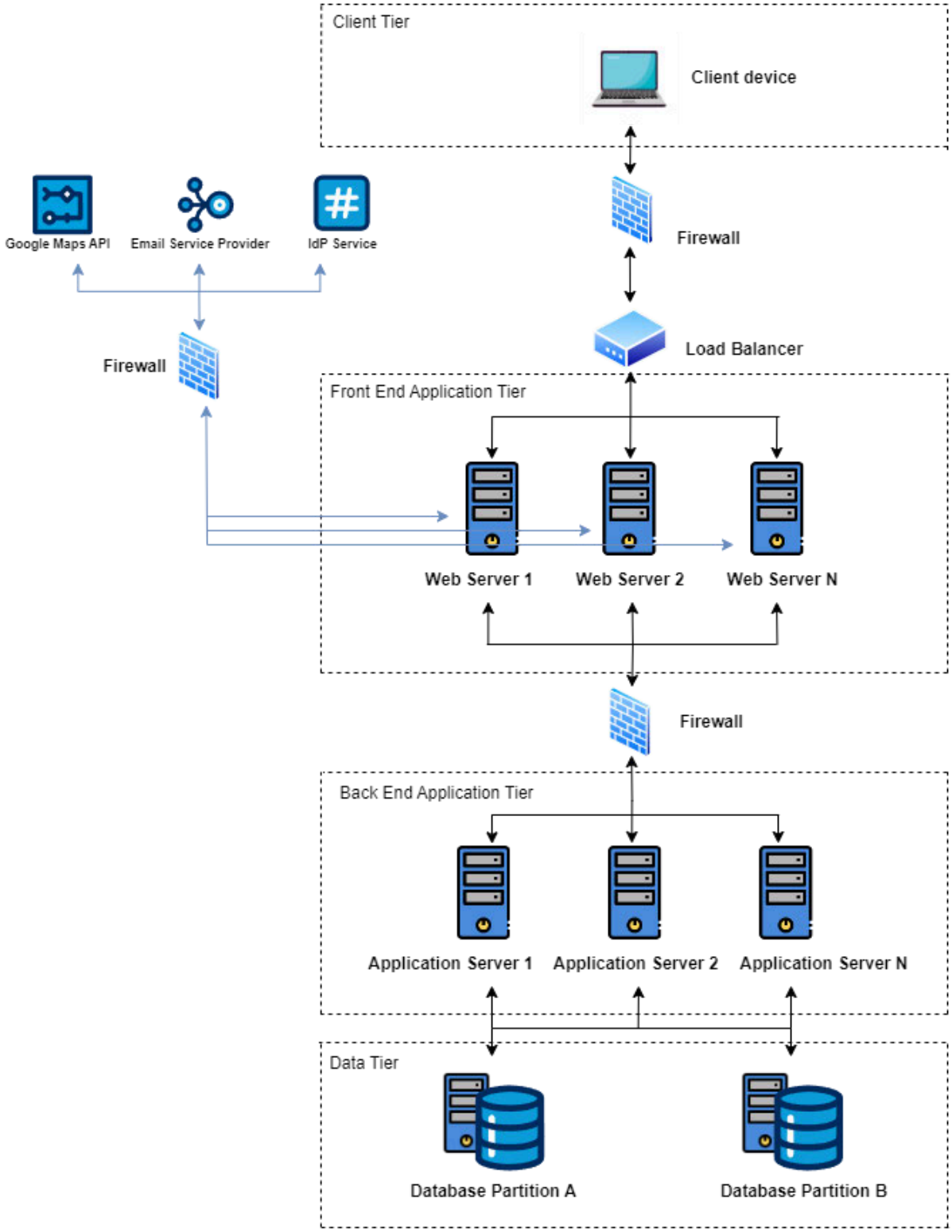
Application Tier - Back End

The back end application tier is responsible for hosting the services offered by the system and for addressing all the user requests that need the retrieval of data or any kind of computation.

The architecture of this tier is organized in microservices, which are replicated and distributed across different machines to ensure availability. These services interact with one another through APIs that each component exposes. Communication between them is facilitated via REST calls, allowing for seamless integration and data exchange across the various components.

Data Tier

The data tier is crucial for managing the system's data and providing application components with efficient access. It consists of a series of replicated databases that must maintain consistency and availability. Each database is partitioned to align with the specific microservice it supports. This means that each microservice has access to dedicated, easily retrievable data storage pertinent to its functions. This approach ensures the decoupling of functionalities extends to the data layer, enhancing the system's efficiency and reliability by allowing each microservice to independently access and manage its data.



Full Architecture View

2.2 Component view

The system consists of various components, including some for the front end, some for the back end, and each component performs a specific service. Additionally, it will require interaction with some external components.

Back-end components

- **User Management Service**

This component manages user profiles, enabling users to modify the information they wish to display, such as locations, textual descriptions, and uploaded documents. Students can upload their CVs, while companies manage projects and related files. When a new location is added by the user, the component interacts with the Maps external service to retrieve and store the related coordinates. When a new file is uploaded, the component interacts with the Document Analyzer and the Recommender System to identify the most suitable users for that file. Subsequently, it calls the Notification Service to inform the relevant users. Additionally, this component allows users to view the profiles of others.

- **Explore Service**

The Explore Service facilitates the system's search functionality, allowing users to adjust filters to find preferred job opportunities for both students and companies. It communicates with the Recommender System in case the user decides to prioritize results based on recommendations.

- **Messaging Service**

This component manages user-to-user communication. It integrates with the Notification Service to send email alerts when a message is received.

- **Internship Management Service**

This component oversees the progression of selection processes and ongoing internships. It enables users to perform actions relevant to different stages of the internship, such as handshakes, scheduling interviews, proposing contracts, submitting complaints, and terminating contracts. Additionally, it works with the Notification Service to inform users about updates regarding internships they are involved in.

- **Organization Registration Service**

This component handles the registration of organizations, including universities and companies. It provides methods for organizations to submit registration requests by uploading required legal documents. Administrators can review pending requests and either accept or reject them. Upon approval, the component creates a new organization user in the database.

- **Document Analyzer Service**

The Document Analyzer processes uploaded files, such as CVs and project documents, to extract the related skill set. It also provides tips and suggestions to users by utilizing statistical data retrieved from the database to feed its algorithms. This component can also convert a textual prompt, given as a filter, into a skill set, in order to refine a research.

- **Recommender System**

This component generates recommendations by identifying suitable projects for students and matching students with company projects. Recommendations are based on skill set compatibility (analyzed by the Document Analyzer) and user ratings from satisfaction surveys; the recommendations are also filtered based on the maximum distance selected by a student or by a company in the context of a project. Each student-project pair is assigned a numerical score representing their compatibility (0 if the distance ranges are violated). The system also triggers notifications for users when a new CV or project file is uploaded, provided the recommendation score exceeds a predefined threshold.

- **Notification Service**

The Notification Service processes notification requests from other components. It generates email content based on parameters provided by the requesting components and sends notifications through the Email Service Provider.

- **Authentication Service**

This component manages user login processes. It redirects users to their organization login page. For first-time student logins, it creates a corresponding student user profile in the database.

- **DBMS**

This component is responsible for acting as an intermediary between the other components and the database, enabling the retrieval of saved data or the storage of new data.

Front-end components

- **Web App**

This is the component through which users directly interact, providing them with the application's view. This component triggers requests that are then routed to the API gateway. It is important to note that the web app will have different functionalities and interfaces for the three types of users: students, companies, and universities. We will use the term "organization" to refer to both universities and companies.

- **Admin Dashboard**

This component allows the S&C administrators to have privileged access to the system via SSH. It is also able to retrieve the documents submitted by organizations that performed a registration request, and to accept or refuse it.

- **External services**

The following components are external to our system and are treated in a generic manner, without detailing their specific providers. Note that, as specified in section 2.7, requests to these components pass through the proxy acting as an intermediary (this will not be represented in the sequence diagrams).

- **Identity Provider**

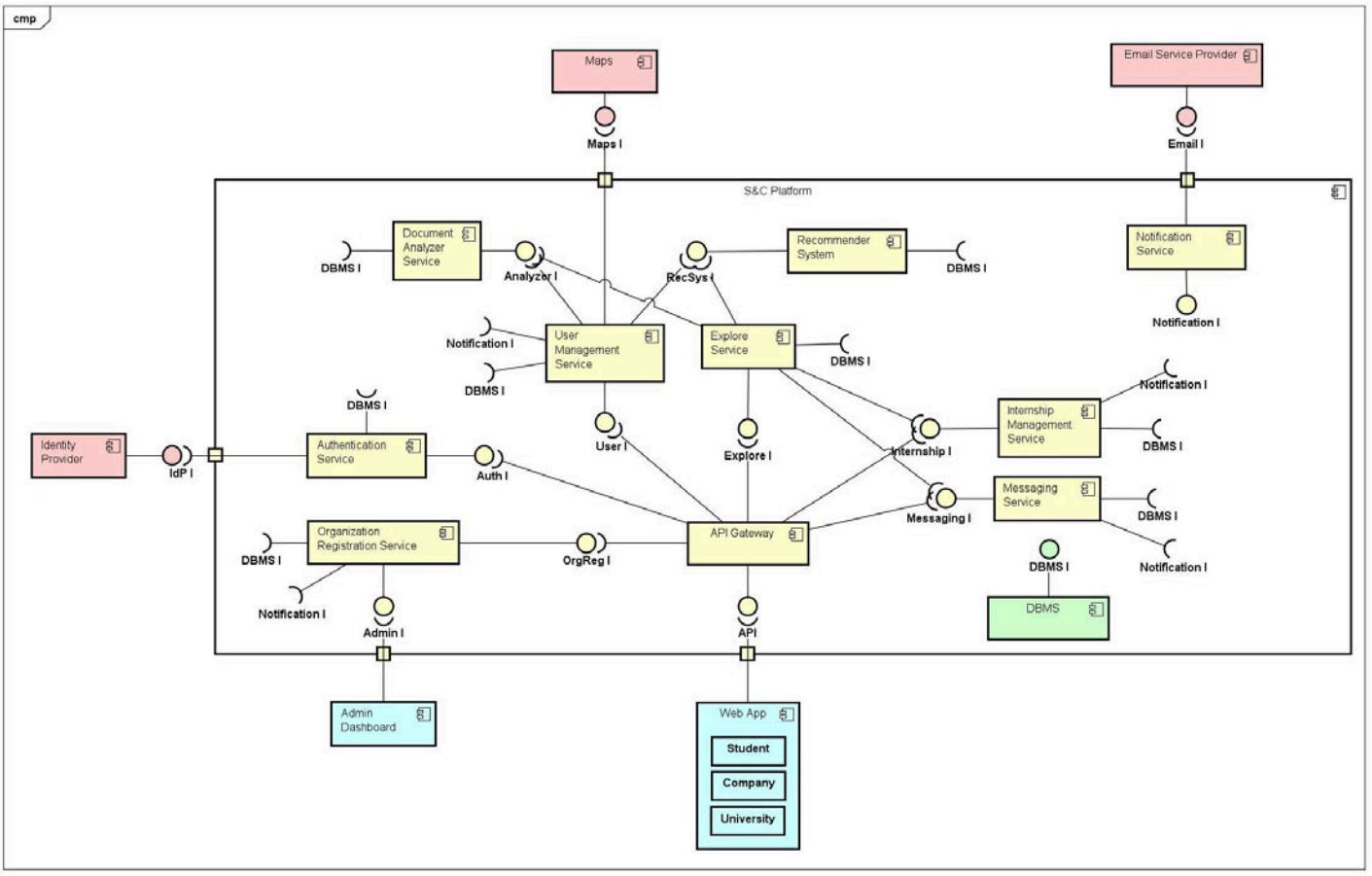
The role of the Identity Provider is that of managing and verifying the identities of users of the S&C platform that are already registered with a university or a company. When a user attempts to log in, the S&C platform queries the IdP with the domain extracted from the user's email. If the domain matches an organization that has a trust relationship with the IdP, the IdP responds with the organization's authentication endpoint. The user then authenticates through this endpoint, and the organization provides a token to the IdP to confirm the user's identity. The IdP then generates a time-limited, reusable access code, allowing the user to bypass the authentication phase for future logins. This code is used to verify the user's identity, granting them access to the platform. Some of the most popular Identity Providers include Microsoft Azure AD, Okta, and Google Identity Platform. Please note that an organization intending to use the S&C platform and register for it must also register with the identity provider responsible for authenticating its members.

- **Maps**

This service will be used by our system to handle locations. In particular, it will return the coordinates of the locations inserted by the user (if the location exists).

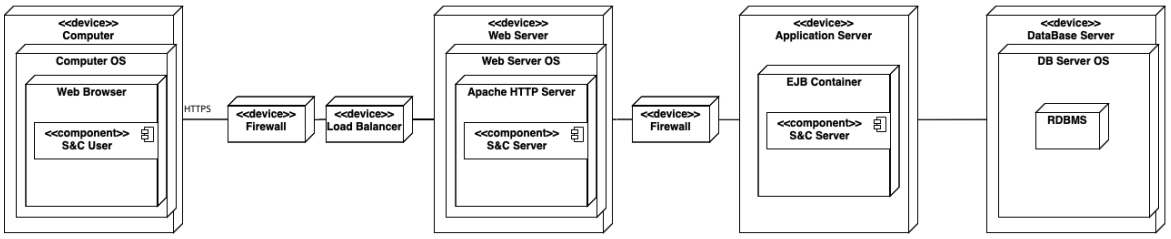
- **Email Service Provider**

This service will interact only with the Notification Service and will allow the system to send email notifications to the users.



2.3 Deployment view

This chapter describes the deployment view of the S&C system. It outlines the execution environment of the system and details the geographical distribution of the hardware components that host the software making up the application.



Further details about the elements in the graph are provided in the following.

Computer

A computer, used by the CPO, only requires a web browser to access the S&C application. The connection is made over HTTPS, passing through the firewall, and is then directed by the load balancer to an available web server for processing.

Firewall

The firewall monitors incoming network traffic, blocking potentially harmful packets before they reach the system. Positioned before the load balancers, it ensures that only safe traffic is allowed into the network, creating a secure perimeter around the system.

Load Balancer

The load balancer distributes incoming requests across multiple web servers to optimize system performance, scalability, and reliability. It ensures an even distribution of traffic, preventing any single server from becoming a bottleneck, and enhances system resilience.

Web Server

The web server handles requests from users accessing the S&C application through a web browser. Upon receiving a request, it forwards it to the application server for processing via an HTTP REST request. If the request requires dynamic content, the web server processes the response and returns the generated web page. For static content, such as the login form, the web server directly responds with previously cached content without involving the application server.

Application Server

The application server is responsible for processing all requests from web servers running the S&C application. It performs the necessary data management and computation and is the only component that communicates directly with the database via TCP/IP.

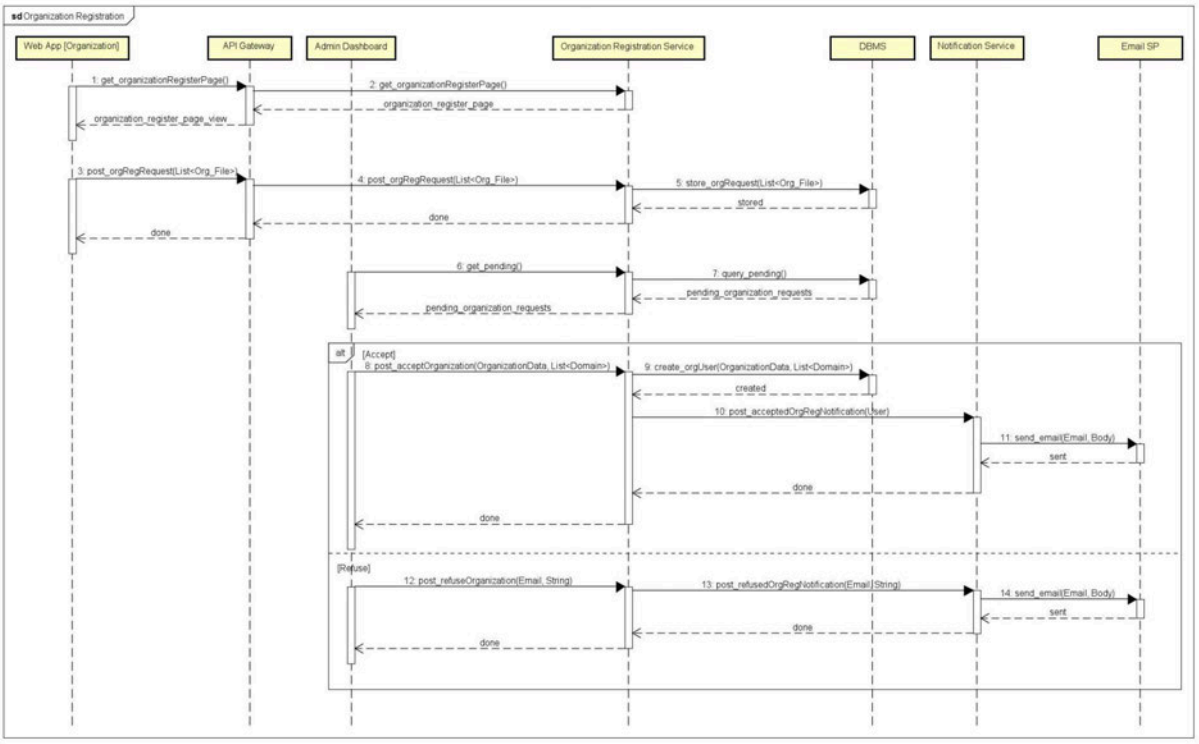
Database Server

The database server uses a clustered system to provide high availability and synchronized data replication. This ensures that the application server always has up-to-date information, reducing errors due to data inconsistencies across database nodes.

2.4 Runtime view

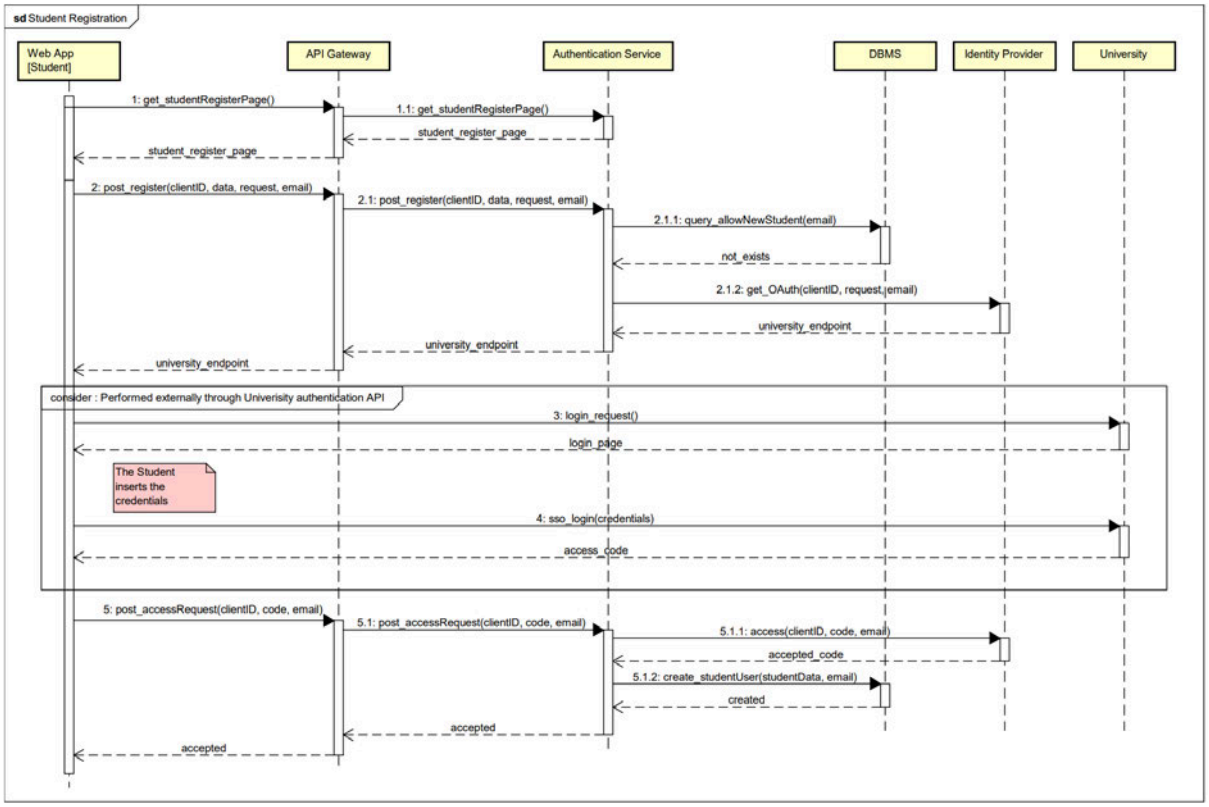
In this section, we will describe the flow of messages between the various components using sequence diagrams for each use case. Please note that, for simplicity, we have omitted the parts that verify, where necessary, whether the requests received are actually authorized for the user making them. For instance, a user can perform only certain actions within the scope of an internship and cannot interact with internships belonging to others, and so on. Additionally, we have excluded possible error messages related to incorrectly specified interface function parameters, issues with external services, and other types of errors.
Moreover, we will not represent that some components, especially the API Gateway, may include a cache and therefore do not require database access for every request. Instead, we will represent the more general case (where every request passes through the database).
While these aspects are critical for the proper functioning of the system, they have been omitted to focus on the main logical flow that leads to the execution of a specific use case.

[UC1] Organization Registration



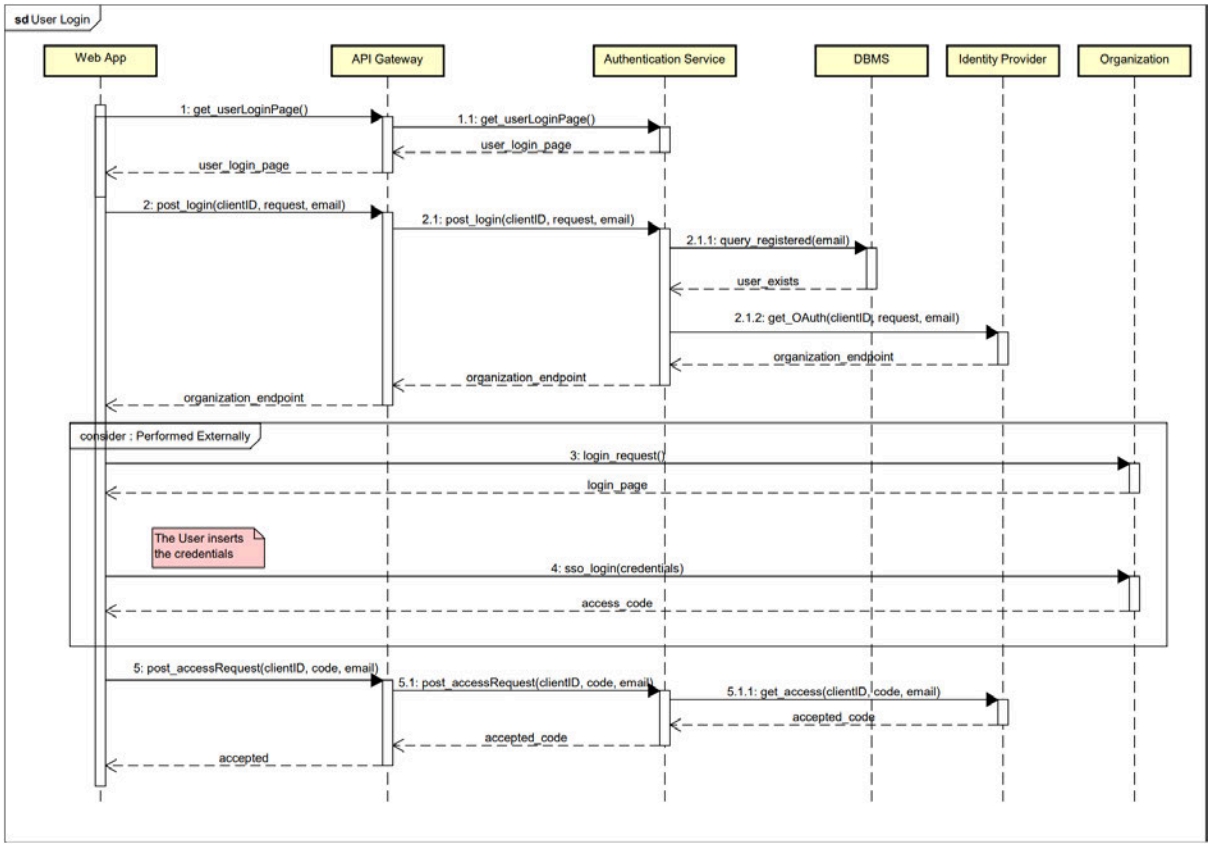
This sequence diagram shows the flow of messages for the Organization Registration use case. The organization uploads a series of legal documents (called “Org_File”) to request the registration; then, the administrators will check for pending requests, will analyze (externally) the reliability of those documents, and will decide whether to accept or to refuse the requests. In case of acceptance, a new organization user is created in the database, with the related domains for future log in of students, for what concerns universities, and managers, for what concerns companies. In both cases, the organization will receive an email notification.

[UC2] Student Registration



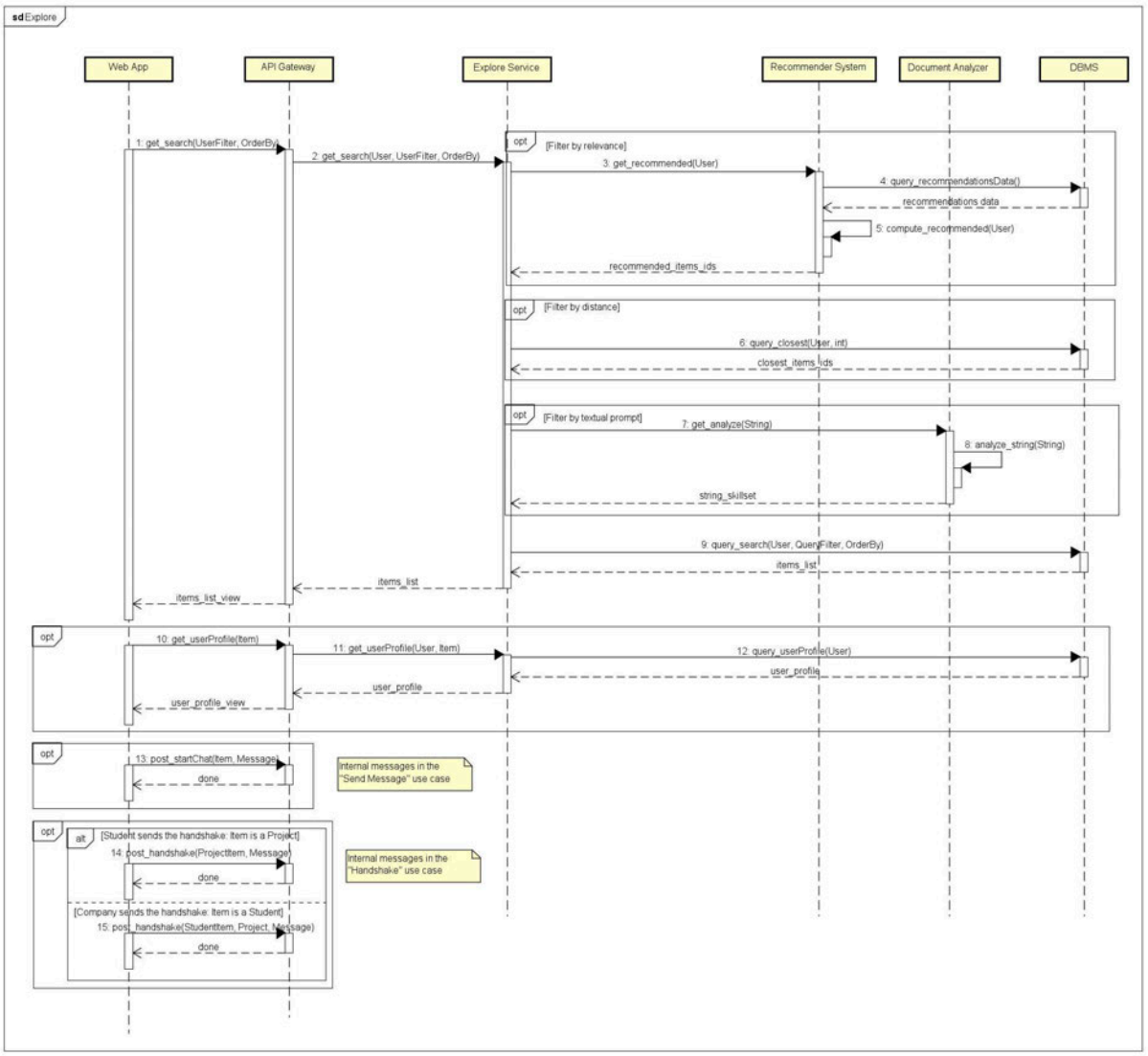
This sequence diagram shows the flow of messages for the Student Registration use case. When a student logs into the platform for the first time, they have to request registration; the system will check whether the user already exists and, if not, it will trigger the authentication phase. After that, a new instance of student user will be created in the database, and a welcome screen will show to the user asking to complete their profile. Since the flow of messages for the profile creation is the same as the one for the profile modification, it will be shown in detail in the “Profile Visualization & Modification” use case.

[UC3] User Login



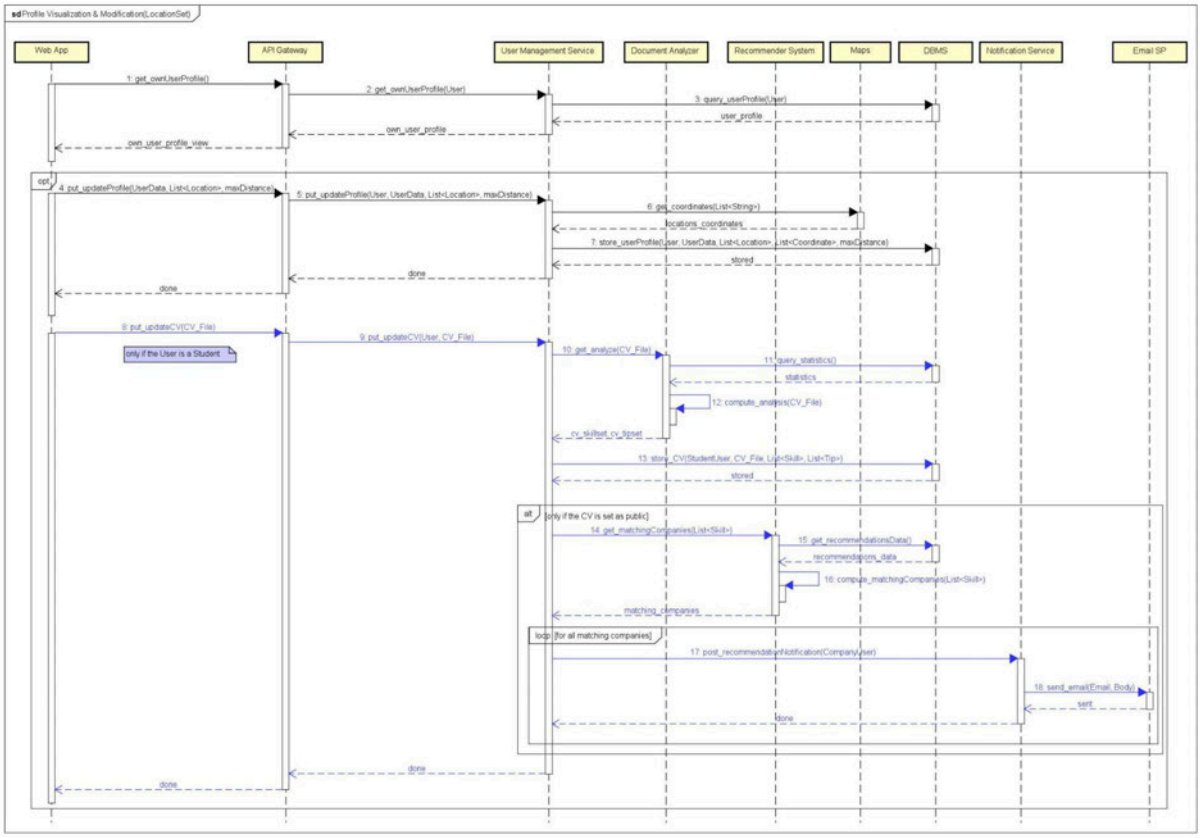
This sequence diagram shows the flow of messages for the User Login use case. The user can choose from three login options based on whether they are a student, a university representative, or a company representative. While the pages displayed to the user differ slightly depending on their role, the login process itself remains consistent. Therefore, for simplicity, the login procedure is illustrated for a generic user. The role of the identity provider is explained in detail in the “Component View” section.

[UC4] Explore



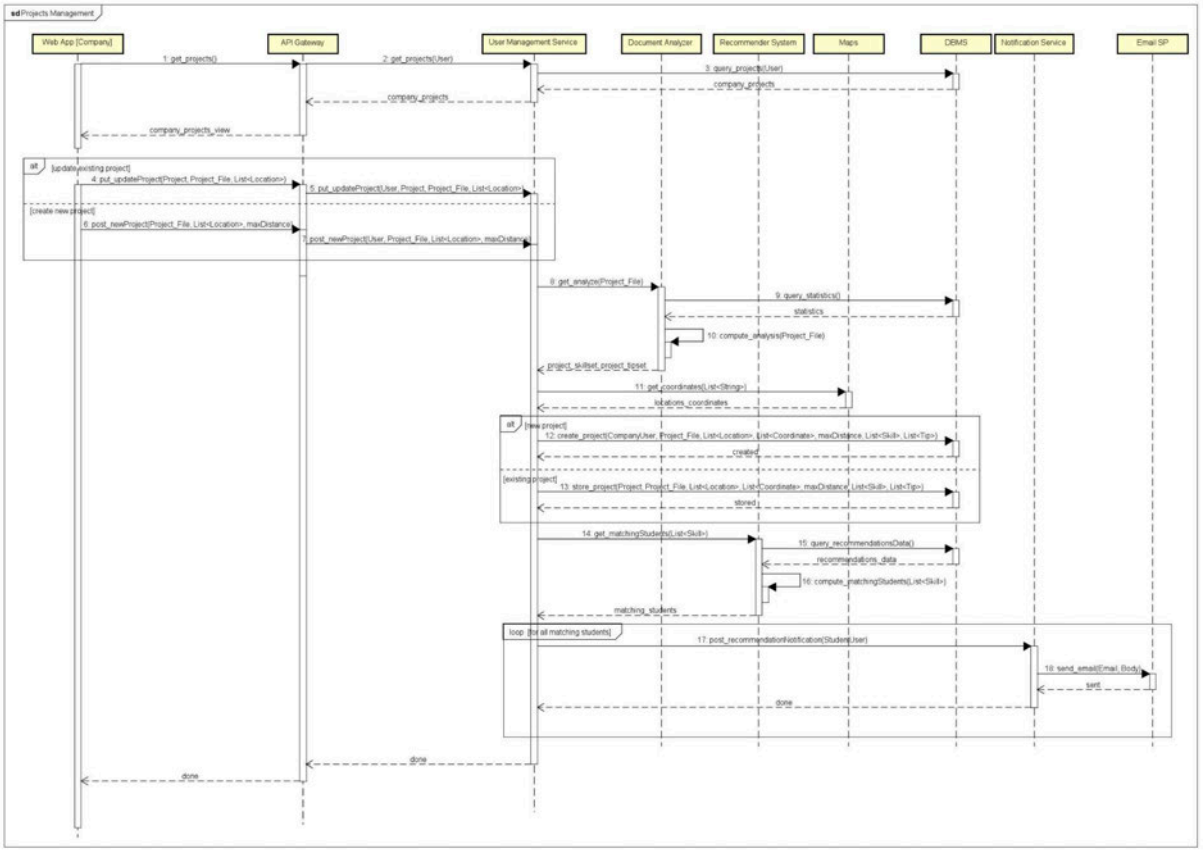
This sequence diagram shows the flow of messages for the Explore use case. The user can decide to adjust a set of filters, and the way the results will be ordered. If the user wants to base the results on recommendations, the recommender system will compute the suitable item ids based on the data stored on the DB; if the user wants to apply a distance filter, the Explore Service will perform a query to the database retrieving the ids of the items in the range; if the user wants to filter by a textual prompt, describing the skills they're looking for, the text will be analyzed by the Document Analyzer to return the associated skill set. These ids and this skill set will be part of the final filter with which the Explore Service will query the database. After performing the search, the user can select an item and view the corresponding user profile, start a chat, or send a handshake. The latter two cases are detailed in the sequence diagrams of the related use cases.

[UC5] Profile Visualization & Modification



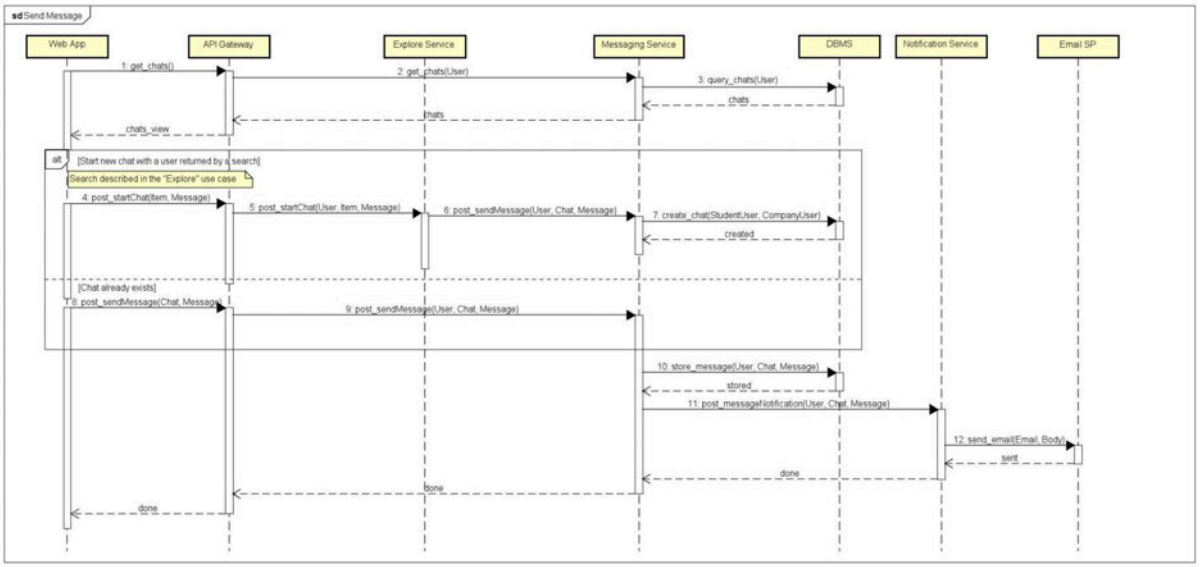
This sequence diagram shows the flow of messages for the Profile Visualization & Modification use case. In addition to viewing their profile, users will be able to update it by adding new data, and in the case of students, uploading a new CV. If new locations are added, the Maps service will be consulted to retrieve their coordinates. The CV will be analyzed by the Document Analyzer to extract the corresponding set of skills and set of tips, which will then be saved in the database along with the other data. The recommender system will calculate which companies have projects with a skill set that sufficiently matches the one just computed and will send a notification to these companies, recommending the student. This process only takes place if the student has set their profile to public.

[UC6] Projects Management



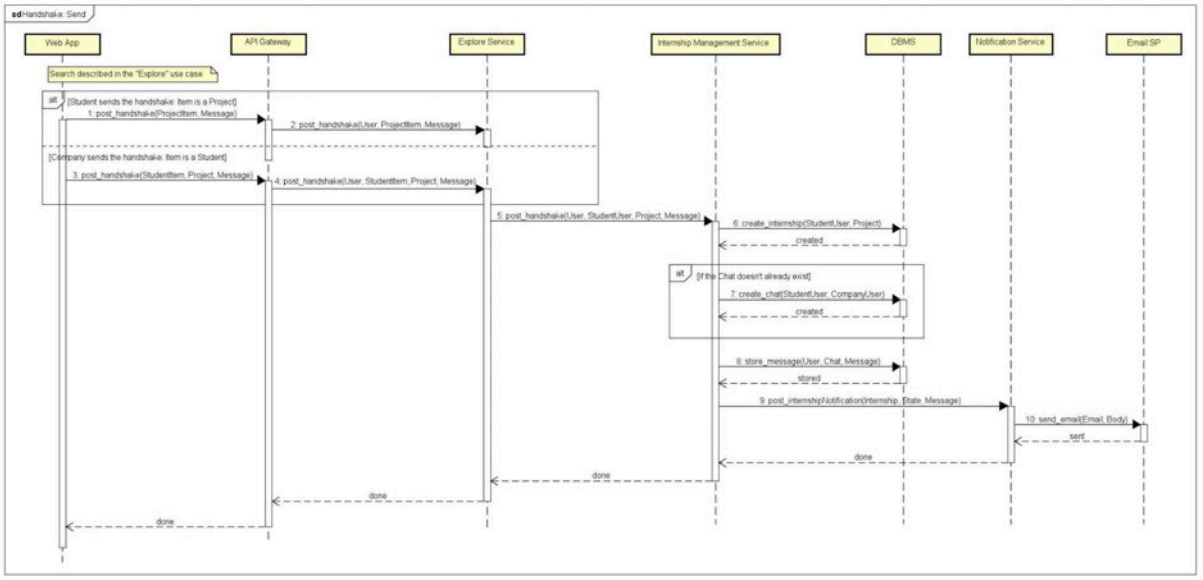
This sequence diagram shows the flow of messages for the Projects Management use case. Companies, in addition to viewing their existing projects, will be able to update the information for current projects or create new ones. Each project must include, among the provided data, a file describing the required skills, the locations associated with the project, and the maximum geographical distance from these locations within which they want to receive notifications about students. The skill set is extracted by the Document Analyzer, while the coordinates of the locations are provided by the Maps service. The recommender system will calculate which students have a skill set that sufficiently matches the one just computed and will send a notification to these students, recommending the project.

[UC7] Send Message

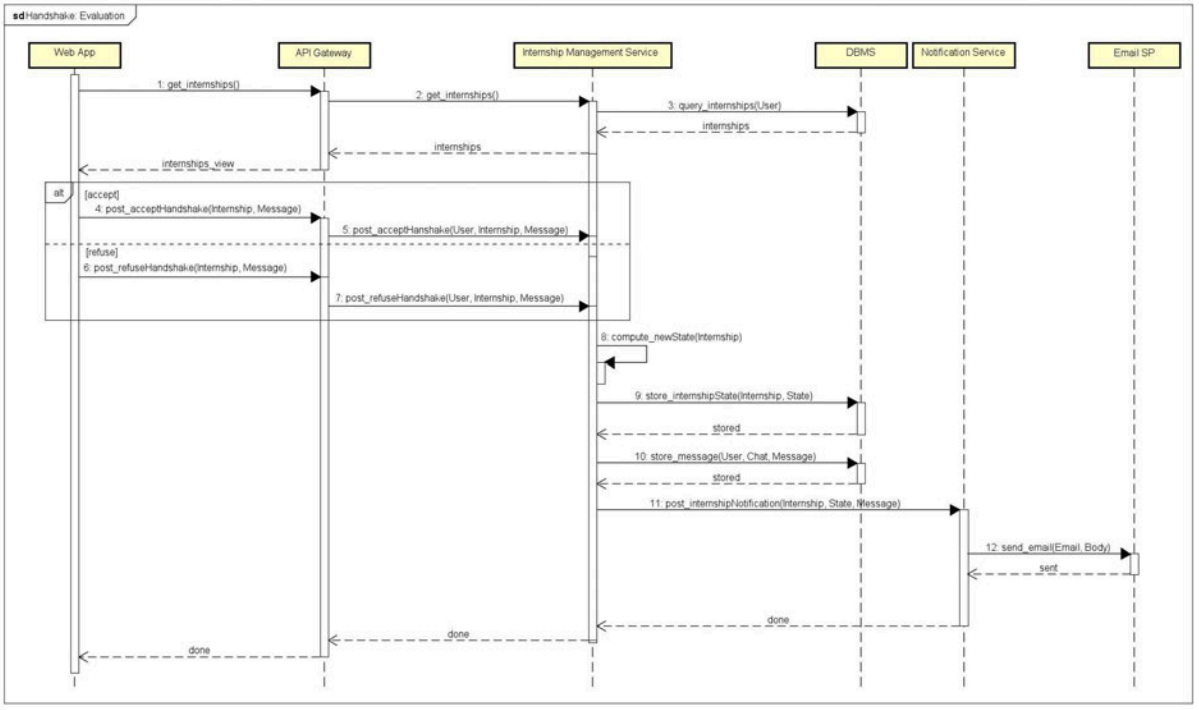


This sequence diagram shows the flow of messages for the Send Message use case. Users can view existing chats along with their messages and send new messages. The recipient will receive an email notification triggered by the notification service. If a user wants to start a chat with another user without an internship already established between them (Step 0), they can do so by initiating a search (the "Explore" use case) and choosing to start a chat with one of the items returned by the search.

[UC8] Handshake

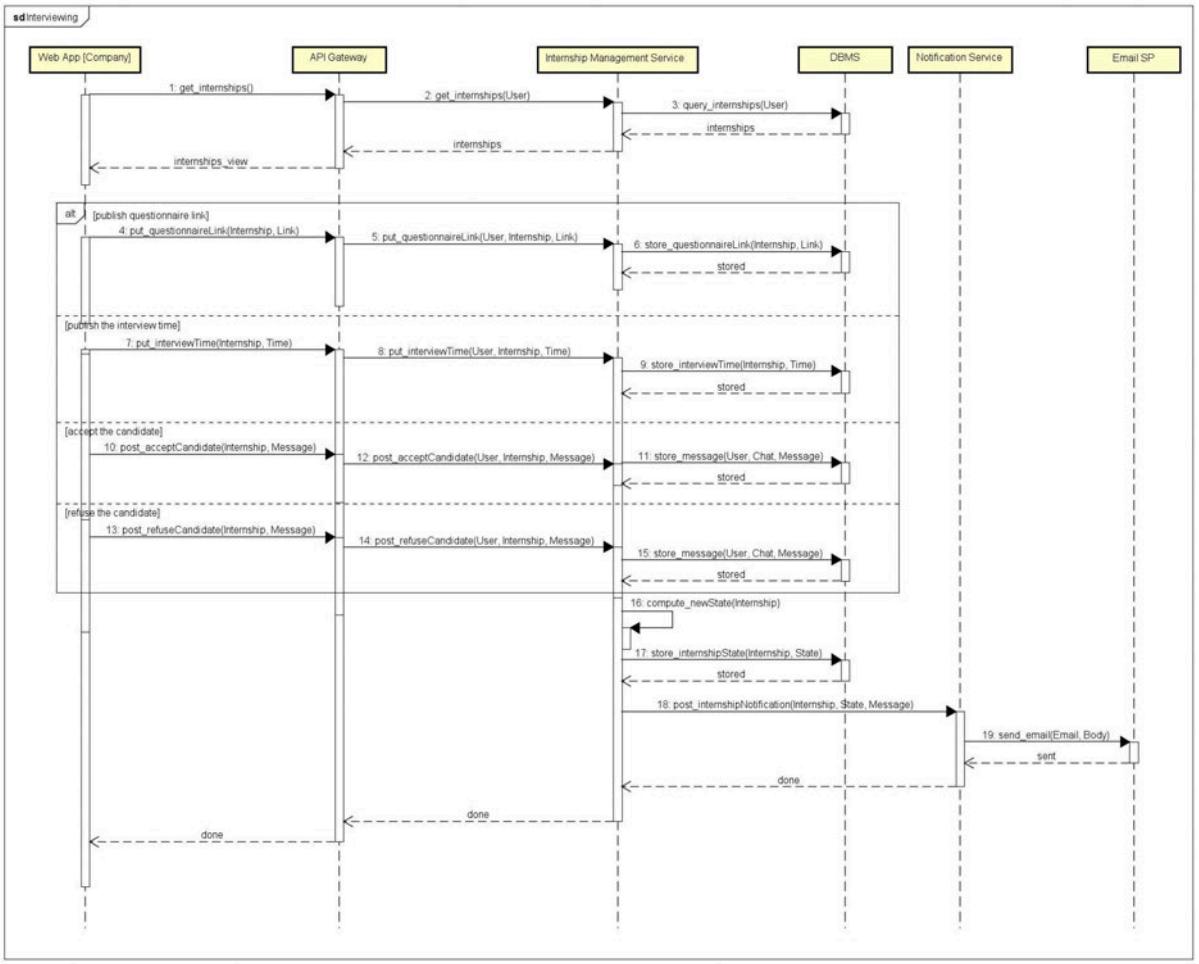


This sequence diagram shows the flow of messages for the Handshake use case. A user, after performing a search, can send a handshake to one of the items returned in the results. If the user is a student, the items returned will be projects; if the user is a company, the items will be students, and the company must also specify which project they are contacting the student about.



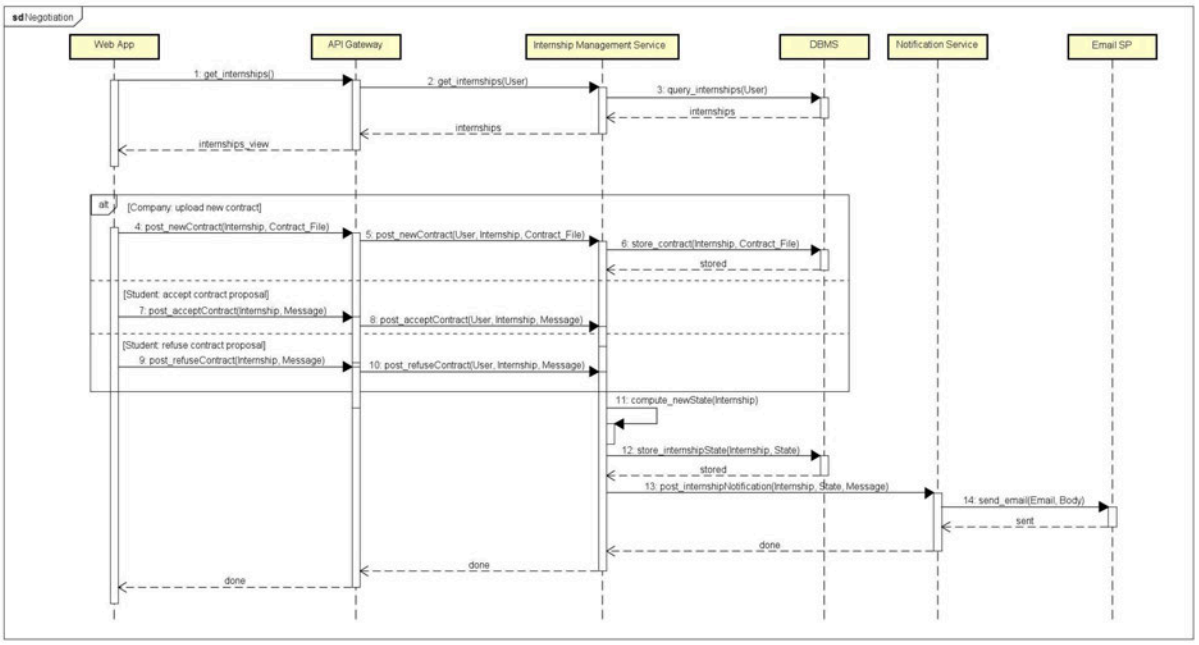
The message attached to the handshake is saved in the message database. A notification is sent to the recipient of the handshake, who can then respond by accepting or declining it, including a message. Also in this case, an email notification is also sent to the other party.

[UC9] Interviewing



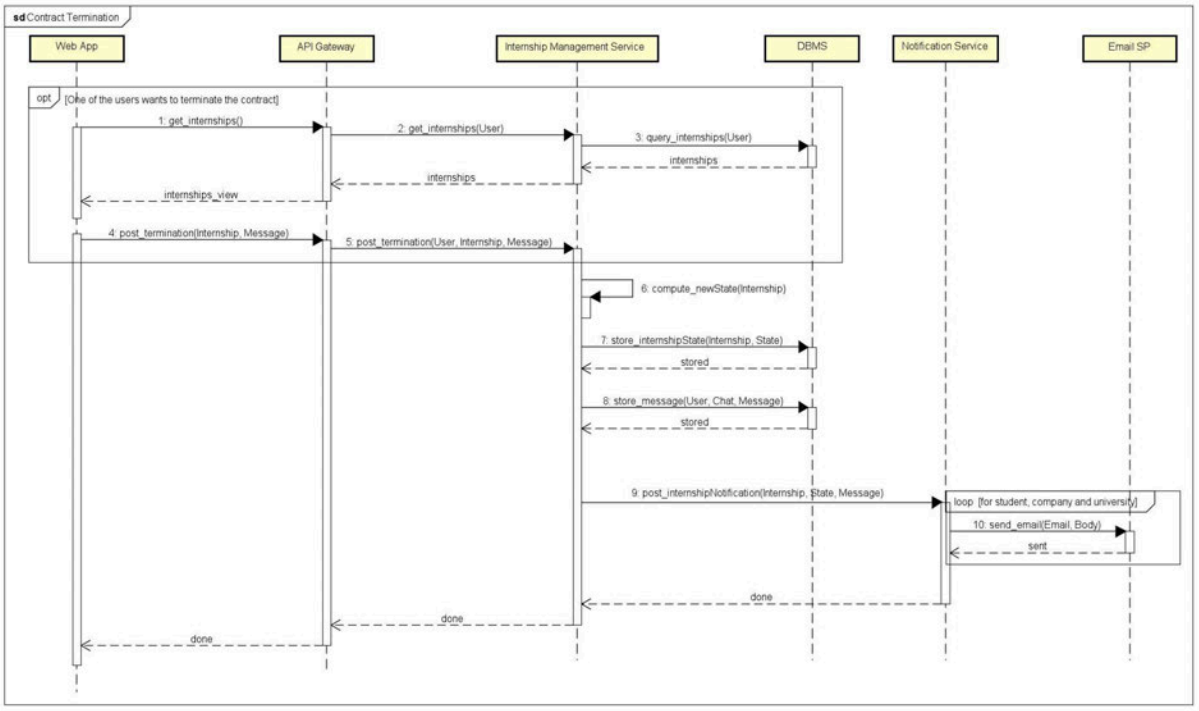
This sequence diagram shows the flow of messages for the Interviewing use case. At this stage, the company can initially choose to send a link to a questionnaire (prepared and managed externally to the system) for the student to complete before the actual interview, or proceed directly with the interview by publishing its schedule. After the interview, the company can decide whether to accept or reject the candidate. All these actions trigger the Internship Management Service to compute the new internship status, save it in the database, and send an email notification to the involved parties through the Notification Service and the Email Service Provider.

[UC10] Negotiation



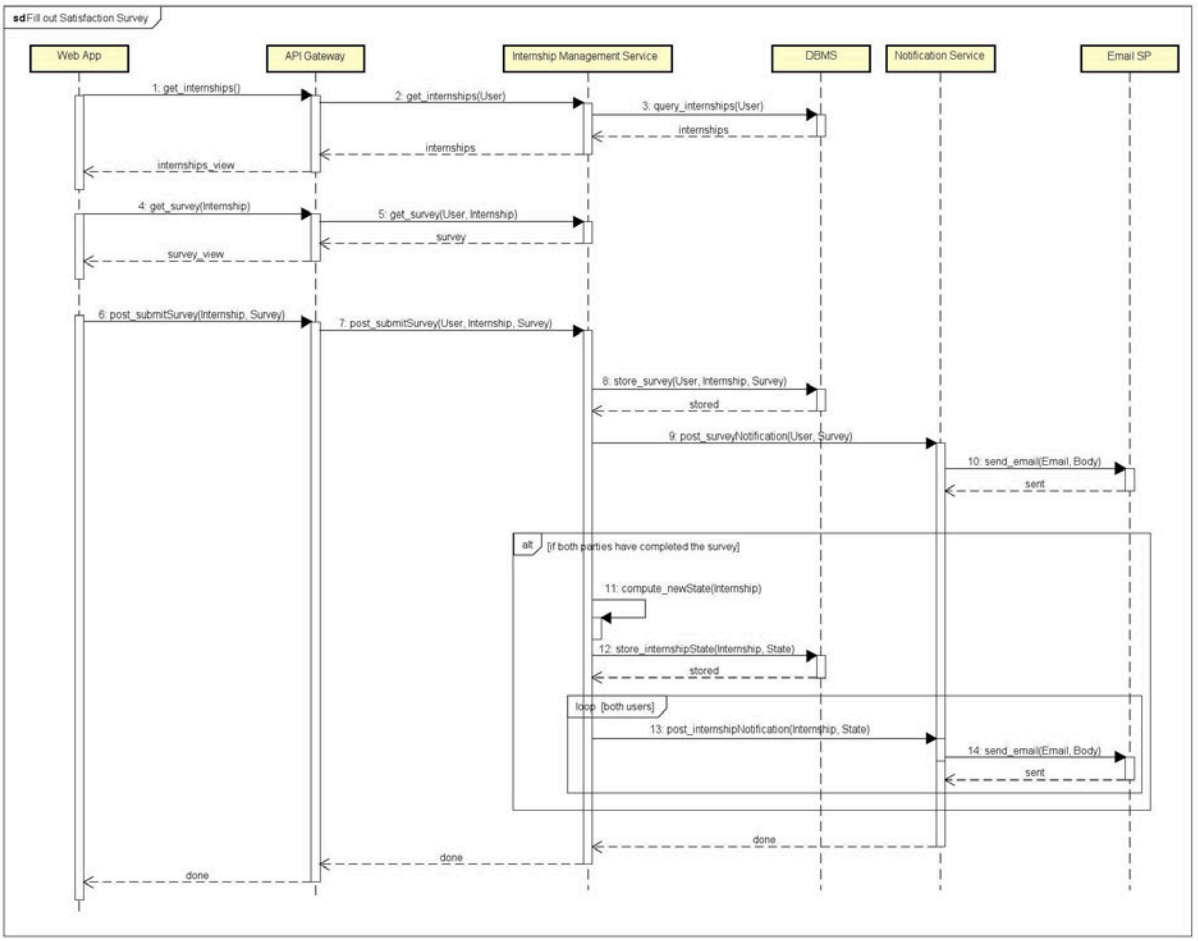
This sequence diagram shows the flow of messages for the Negotiation use case. Companies can upload a new contract, while students, once they receive a contract, can choose to accept it, contact the company via message to negotiate the terms, or reject it. In all these cases, the Internship Management Service will update the internship's status, save the new state in the database, and send a notification to the other party through the Notification Service and the Email Service Provider.

[UC11] Contract Termination



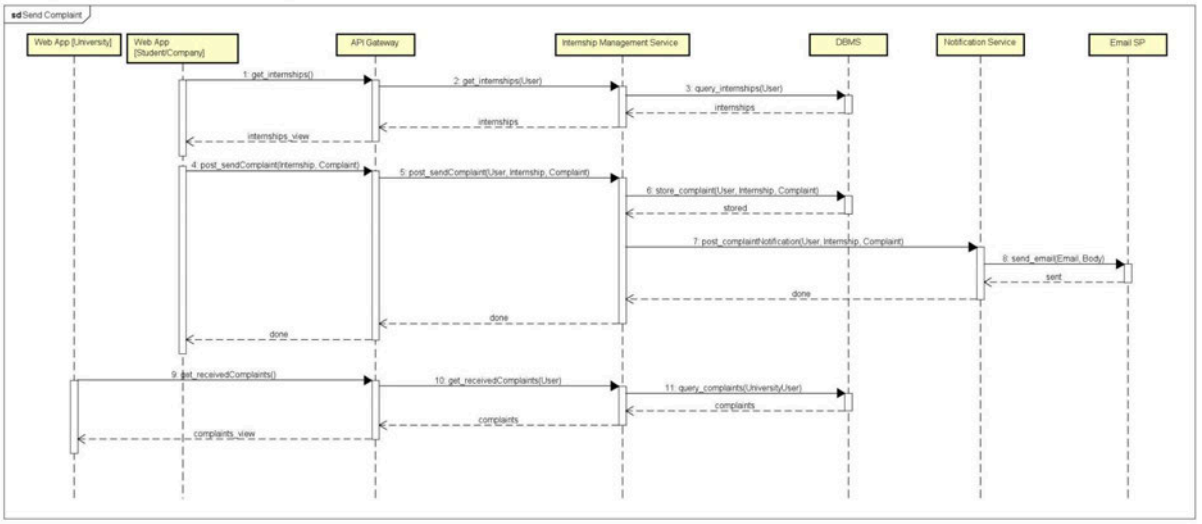
This sequence diagram shows the flow of messages for the Contract Termination use case. If a user wants to terminate an internship, they must select the internship, choose the option to end the contract, and attach a message. The Internship Management Service will compute the new internship status (making the satisfaction survey available), save the updated status in the database, and send a notification to the involved parties, including the student's university, via the Notification Service and the Email Service Provider. This process applies both when one of the parties decides to terminate the internship and when the internship reaches its natural conclusion.

[UC12] Fill out Satisfaction Survey



This sequence diagram shows the flow of messages for the Fill out Satisfaction Survey use case. After the conclusion of an internship, both parties must complete the satisfaction survey: they select the internship, fill out the survey that will already be available, and once each party completes it, an email notification will be sent to the other party via the Notification Service and the Email Service Provider, informing them and forwarding the responses. Once both parties have completed the survey, the Internship Management Service will update the internship's status to “completed”, save the new state in the database, and notify the involved parties.

[UC13] Send Complaint



This sequence diagram shows the flow of messages for the Send Complaint use case. A user who wants to submit a complaint must select the corresponding internship, write a message explaining the reasons for the complaint, and an email notification will be sent through the Notification Service and the Email Service Provider to the university of the student involved in the internship. The university handles the complaint externally to the system.

2.5 Component interfaces

This section provides a summary of all the methods within the interfaces of our system. All internal system components will rely on REST API interfaces. Note that the methods have been named - arbitrarily - with an initial prefix indicating the corresponding operation type (get/put/post). Additionally, note that for some methods, overloading has been used, meaning the implementation may vary slightly depending on the parameters provided. For external components, we have assumed the presence of only a few methods that are likely to be available, regardless of the provider.

The names of the classes used as method parameters were chosen to be as self-explanatory as possible. However, to avoid ambiguity, note the following:

- “User” represents a generic user, while “StudentUser”, “CompanyUser”, and “UniversityUser” are subclasses of User and are used when the parameter needs to specify a particular type of user.
- “Item” represents an object returned after a search in the “Explore” section. Similarly, “StudentItem” and “ProjectItem” are subclasses of Item and are used when specificity is required. For example, when a student performs a search, the result will be a ProjectItem, whereas if the search is performed by a company, the result will be a set of StudentItems.

- Classes corresponding to files are all named “<Category>_File”. These classes represent files, potentially with additional metadata related to their specific category.
- “maxDistance” is essentially an integer representing the maximum distance from each selected Location that determines the range for which the user wishes to receive recommendations. For instance, if I am a student and select Vancouver as location with maxDistance = 100, I will receive notifications about new eligible projects located no more than 100 km from Vancouver's coordinates.
- During the search phase, there is a class called “UserFilter”, representing the user's choices for search filters, and a class called “QueryFilter”, representing the actual filter used in the query to the DBMS.

API

- get_chats()
- get_internships()
- get_ownUserProfile()
- get_userProfile(Item)
- get_organizationRegister()
- get_projects()
- get_receivedComplaints()
- get_search(UserFilter, OrderBy)
- get_studentRegisterPage()
- get_survey(Internship)
- get_userLoginPage()
- post_acceptCandidate(Internship, Message)
- post_acceptContract(Internship, Message)
- post_acceptHandshake(Internship, Message)
- post_accessRequest(ClientID, Code, Email)
- post_handshake(ProjectItem, Message)
- post_handshake(StudentItem, Project, Message)
- post_login(ClientID, Request, Email)
- post_newContract(Internship, Contract_File)
- post_newProject(Project_File, List<Location>, maxDistance)
- post_orgRegRequest(List<Org_File>)
- post_refuseCandidate(Internship, Message)
- post_refuseContract(Internship, Message)
- post_refuseHandshake(Internship, Message)
- post_register(ClientID, Data, Request, Email)
- post_sendComplaint(Internship, Complaint)
- post_sendMessage(Chat, Message)
- post_startChat(Item, Message)
- post_submitSurvey(Internship, Survey)
- post_termination(Internship, Message)
- put_interviewTime(Internship, Time)
- put_questionnaireLink(Internship, Link)
- put_updateCV(CV_File)

- put_updateProfile(UserData, List<Location>, maxDistance)
- put_updateProject(Project, Project_File, List<Location>)

Auth I

- get_studentRegisterPage()
- get_userLoginPage()
- post_accessRequest(ClientID, Code, Email)
- post_login(ClientID, Request, Email)
- post_register(ClientID, Data, Request, Email)

User I

- get_projects(User)
- get_ownUserProfile(User)
- post_newProject(User, Project_File, List<Location>, maxDistance)
- put_updateCV(User, CV_File)
- put_updateProfile(User, UserData, List<Location>, maxDistance)
- put_updateProject(User, Project, Project_File, List<Location>)

OrgReg I

- get_organizationRegisterPage()
- get_pending()
- post_acceptOrganization(Organization, List<Domain>)
- post_orgRegRequest(List<Org_File>
- post_refuseOrganization(Email, String)

Explore I

- get_search(User, UserFilter, OrderBy)
- get_userProfile(User, Item)
- post_handshake(User, ProjectItem, Message)
- post_handshake(User, StudentItem, Project, Message)
- post_startChat(User, Item, Message)

Messaging I

- get_chats(User)
- post_sendMessage(User, Chat, Message)

RecSys I

- get_matchingCompanies(List<Skill>)
- get_matchingStudents(List<Skill>)
- get_recommended(User)

Analyzer I

- get_analyze(CV_File)
- get_analyze(Project_File)
- get_analyze(String)

Internship I

- get_internships(User)
- get_receivedComplaints(User)
- get_survey(User, Internship)
- post_acceptCandidate(User, Internship, Message)
- post_acceptContract(User, Internship, Message)
- post_acceptHandshake(User, Internship, Message)
- post_handshake(User, StudentUser, Project, Message)
- post_newContract(User, Internship, Contract_File)
- post_refuseCandidate(User, Internship, Message)
- post_refuseContract(User, Internship, Message)
- post_refuseHandshake(User, Internship, Message)
- post_sendComplaint(User, Internship, Complaint)
- post_submitSurvey(User, Internship, Survey)
- post_termination(User, Internship, Message)
- put_interviewTime(User, Internship, Time)
- put_questionnaireLink(User, Internship, Link)

Notification I

- post_acceptedOrgRegNotification(User)
- post_complaintNotification(User, Internship, Complaint)
- post_internshipNotification(Internship, State)
- post_internshipNotification(Internship, State, Message)
- post_messageNotification(User, Chat, Message)
- post_recommendationNotification(CompanyUser)
- post_recommendationNotification(StudentUser)
- post_refusedOrgRegNotification(Email, String)
- post_surveyNotification(User, Survey)

DBMS I

Each of the following methods corresponds to one or more specific SQL operations in a parametric form. The prefixes indicate the type of operation: “query” corresponds to data retrieval from the database, “store” updates or adds information to existing objects, and “create” introduces new objects. The actual implementation will, of course, depend on the specific DBMS chosen.

- create_chat(StudentUser, CompanyUser)
- create_internship(StudentUser, Project)
- create_orgUser(OrganizationData, List<Domain>)
- create_project(CompanyUser, Project_File, List<Location>, List<Coordinate>, maxDistance, List<Skill>, List<Tip>)
- create_studentUser(StudentData, Email)
- query_allowNewStudent(Email)
- query_chats(User)
- query_closest(User, int)
- query_complaints(UniversityUser)
- query_internships(User)
- query_pending()
- query_projects(User)
- query_recommendationsData()
- query_registered(Email)
- query_search(User, QueryFilter, OrderBy)
- query_statistics()
- query_userProfile(User)
- store_complaint(User, Internship, Complaint)
- store_contract(Internship, Contract_File)
- store_CV(StudentUser, CV_File, List<Skill>, List<Tip>)
- store_internshipState(Internship, State)
- store_interviewTime(Internship, Time)
- store_message(User, Chat, Message)
- store_orgRequest(List<Org_File>)
- store_project(Project, Project_File, List<Location>, List<Coordinate>, maxDistance, List<Skill>, List<Tip>)
- store_questionnaireLink(Internship, Link)
- store_survey(User, Internship, Survey)
- store_userProfile(User, UserData, List<Location>, List<Coordinate>, maxDistance)

IdP I

- get_OAuth(ClientID, Request, Email)
- get_access(ClientID, Code, Email)

Maps I

- get_coordinates(List<String>)

Email I

- send_email(Email, Body)

2.6 Selected Architectural Styles and Patterns

Client Server 4-Tiered Architecture

The decision of designing a 4 tiered architecture comes from the need of a system that decouples the interaction with the network and the user from the services that it offers, granting flexibility and at the same time security. In fact, this kind of architecture allows the presence of a series of front-end distributed application servers that are responsible for managing the requests coming from the users and dispatching them to the back-end application logic hosted on separate nodes and responsible for managing the core services and for communicating with the distributed databases.

Microservices Architecture

The use of a microservices architecture enables a system like S&C to scale easily and efficiently, without compromising the platform's overall performance. Simultaneously, it ensures careful management of resources and a robust security. This architecture facilitates the use of various small-scale services, which are effectively coordinated through a load balancing mechanism that assigns each new request to an appropriate replica. By isolating and delivering the required functionalities individually through distinct components, the architecture prevents overloading any part of the platform, thereby enhancing its resilience and reliability. Communication between microservices utilizes the REST architectural style and is implemented asynchronously. Ordered message queues are incorporated as a load balancing mechanism to improve efficiency and load management.

Circuit Breakers

The system must grant high resilience and availability among all its services, to do so it must implement circuit breakers, which are a design pattern used to enhance the fault-tolerance of applications. They act by preventing cascade failures deriving from the dependency on resources that may be unavailable or unreliable, and by detecting system inefficiencies that require the administrators' intervention.

State Pattern

The system is responsible for managing Internship and for enabling the interested users to perform some defined actions during each stage of it. The State pattern is a behavioral pattern that allows an object to modify its behaviour when its internal state changes. By applying this pattern for the design of Internship functionalities, we can address the issue of defining which actions a user can or can't perform during a specific step of the internship preparation.

Command Pattern

The command pattern is an effective method for decoupling the actions requested to the system by a user and the actual server-side implementation of those actions. The pattern simplifies requests management by encapsulating requests as distinct command objects. It allows the system to be easily managed and extended, and it supports the aggregation of similar requests and queuing, enabling batch processing.

Publish Subscribe architecture for Recommendations

A publish subscribe architecture is an effective method for handling a component like the recommender system, which is responsible for informing the users about the opportunities of their interest, such as the publication of a student CV for a company, and for providing them with suggestions that could suit their interests. This architecture allows events such as the insertion of a new CV or a new project to be notified to all the users that may be subscribed to a topic relevant for that publication, according to a List<Domain> of preferences that are statistically derived by the system.

2.7 Other Design Decisions

Proxy as Intermediary

The front-end application tier hosts alongside all the other components a proxy server, which is responsible for handling outgoing API requests from the system to external services. Consequently, its tasks comprehend also managing the API responses it receives and forwarding them to the internal service that requires the computation. The use of a proxy server offers several benefits: it can modify requests, transform responses, log interactions, cache data, and conduct security checks before passing requests to the back end application tier.

Security

The addition of an intermediary tier grants a further level of security: in addition to the firewall that separates the front-end of the system to the internet, another firewall is placed between the two tiers responsible for managing the application layer, adding another barrier for external threats. The API gateway, as previously explained, also offers many functionalities that prevent potential attacks to the services such as managing external requests, authenticating users and acting as load balancer.

Administrators Access

Administrators need a secure method to access the system for handling malfunctions or performing routine maintenance. They can achieve this by using Secure Shell (SSH), a

protocol used to securely access remote systems over unsecured networks. An authorized user authenticates using their SSH key, connects to the system through an encrypted channel, and can execute commands and transfer files remotely.

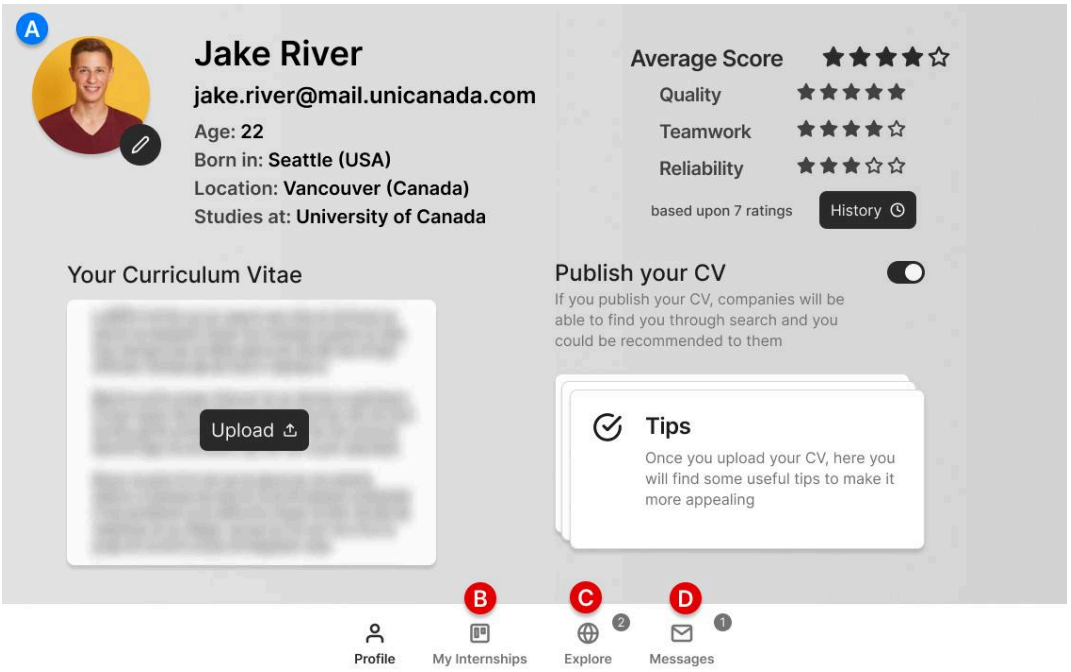
3. User Interface Design

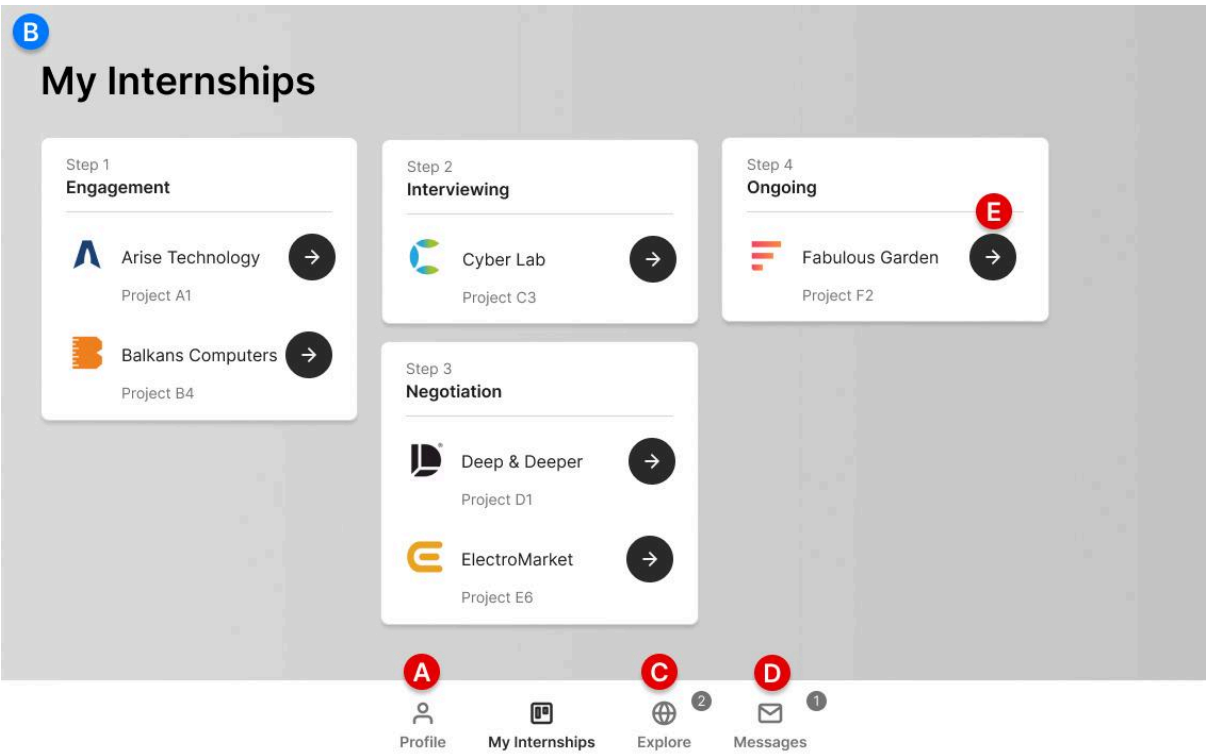
In this chapter, we present a proposed User Interface for our web application. Compared to the RASD, we have highlighted user interaction. Additionally, we have included the university view. As mentioned in the RASD, the company view is not represented, as it is analogous to the student view (with obvious differences regarding profile information and the type of items displayed).

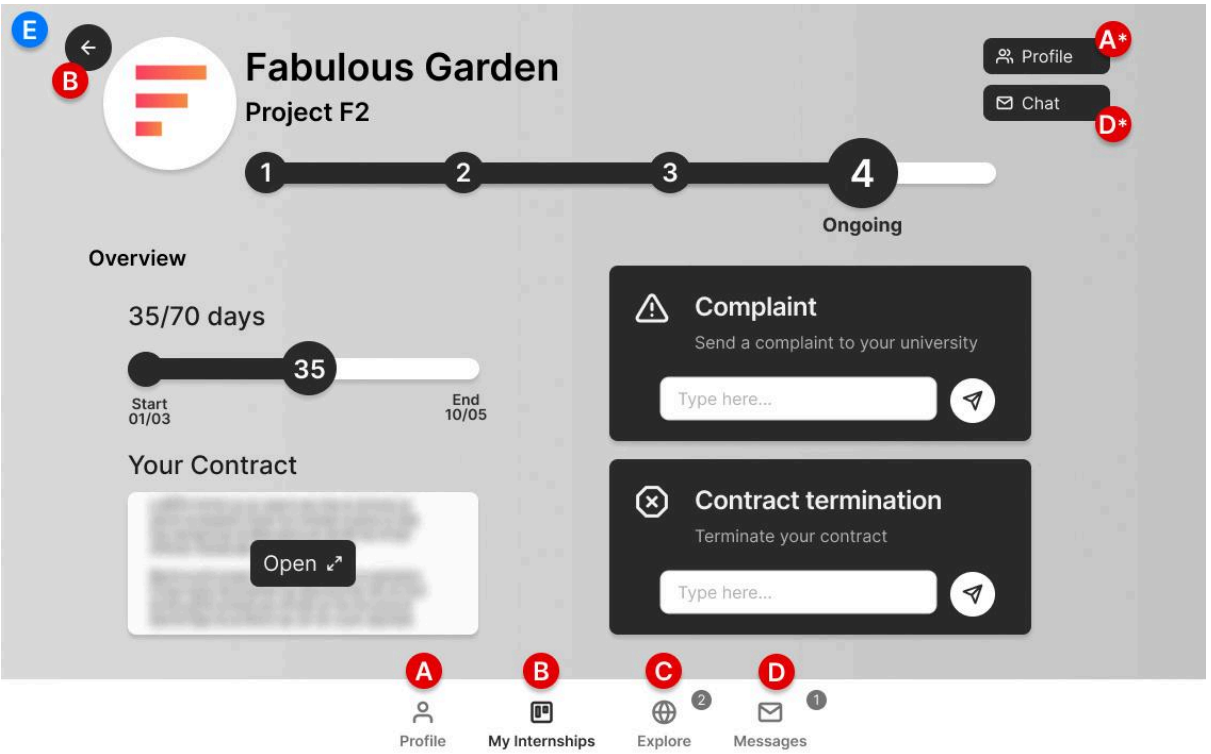
Legend:

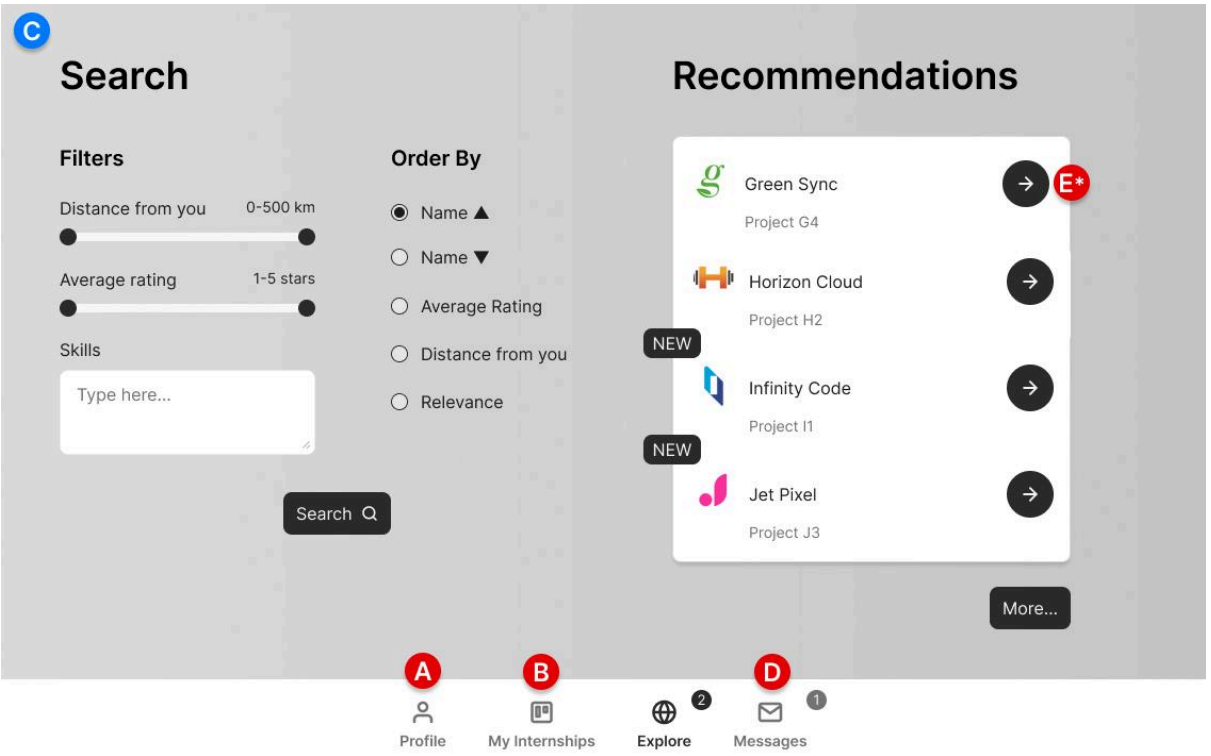
- Red circles with letters: represent a user's click on a specific part of the screen.
- Blue circles with corresponding letters: represent the landing page.
- Red circles with letters followed by asterisks: indicate that the corresponding landing page has a similar layout but is not identical to the one shown. For instance:
    - Each internship will have its own page reflecting its current status;
    - Each user has their own user page. Obviously, when viewing another user's page, the ability to edit data will not be available;
    - The university view of a student's internships may slightly vary;
    - The chat button inside an internship page refers to the chat with the corresponding company;

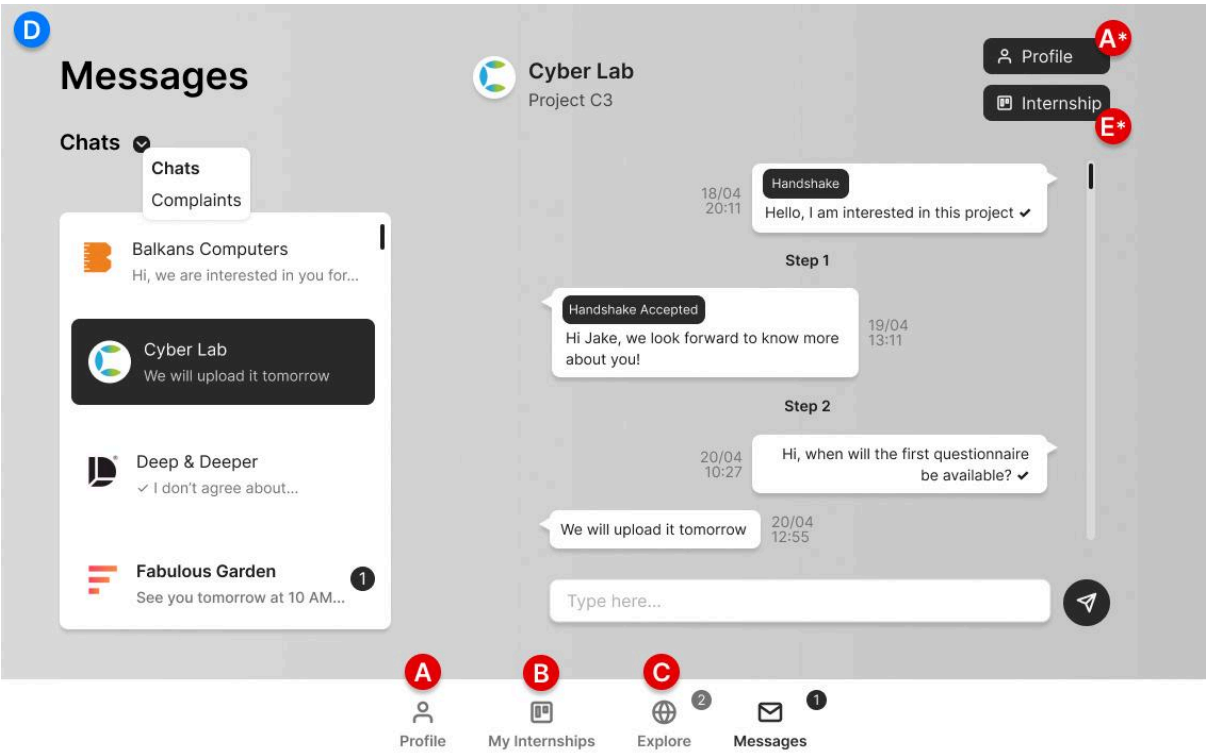
and so on.

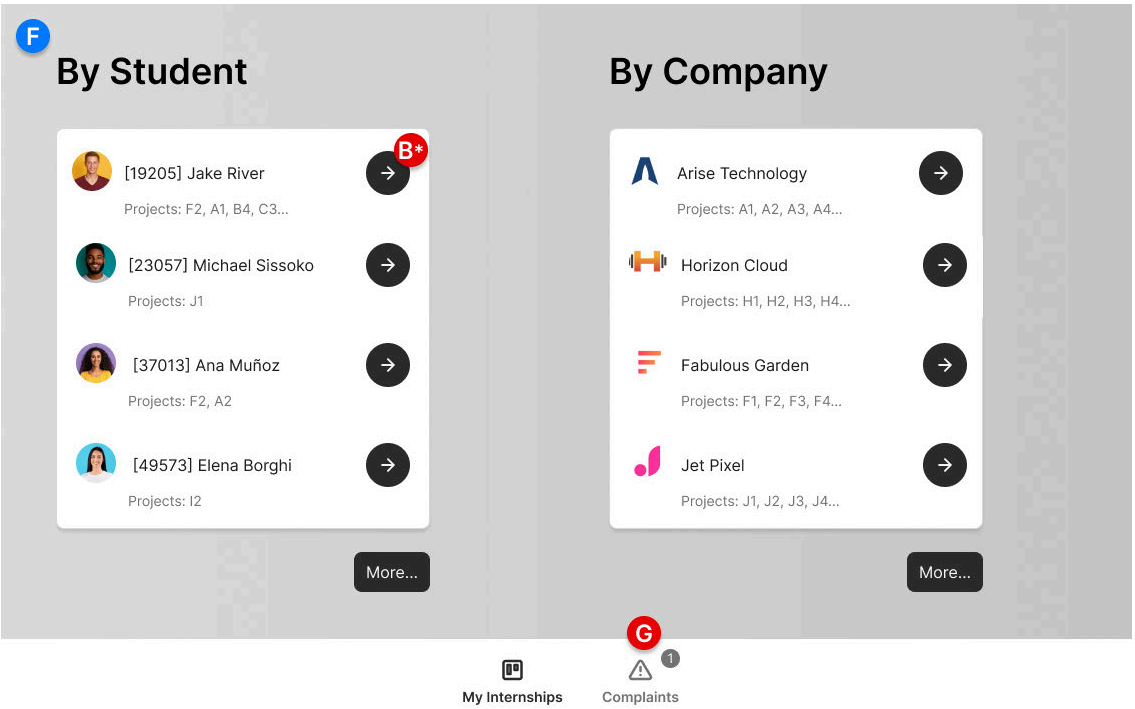


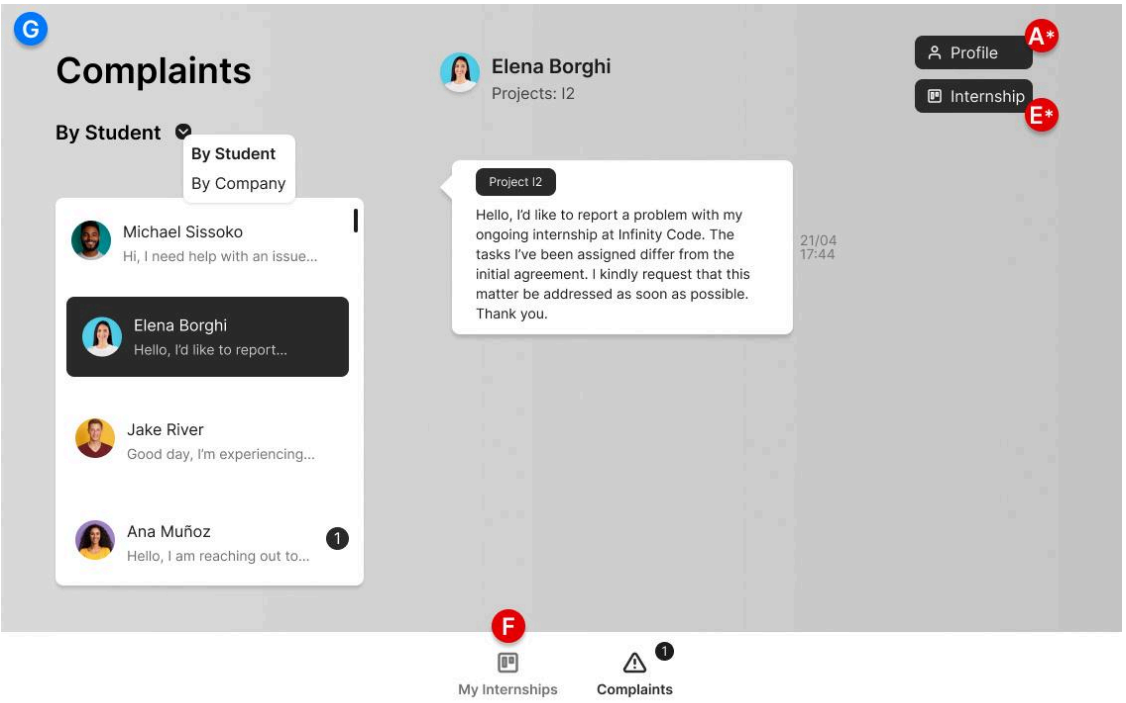

B
My Internships
Step 1
Engagement
Arise Technology
Project A1
Balkans Computers
Project B4
Step 2
Interviewing
Cyber Lab
Project C3
Step 3
Negotiation
Deep & Deeper
Project D1
ElectroMarket
Project E6
Step 4
Ongoing
E
Fabulous Garden
Project F2
A
C
D
Profile
My Internships
Explore
2
Messages
1


E
B
Fabulous Garden
Project F2
Profile
A*
Chat
D*
1
2
3
4
Ongoing
Overview
35/70 days
35
Start
01/03
End
10/05
Your Contract
Open
Complaint
Send a complaint to your university
Type here...
Contract termination
Terminate your contract
Type here...
A
B
C
D
Profile
My Internships
Explore
2
Messages
1


C
Search
Filters
Distance from you
0-500 km
Average rating
1-5 stars
Skills
Type here...
Order By
Name ▲
Name ▼
Average Rating
Distance from you
Relevance
Search
Recommendations
Green Sync
Project G4
E*
Horizon Cloud
Project H2
NEW
Infinity Code
Project I1
NEW
Jet Pixel
Project J3
More...
A
B
D
Profile
My Internships
Explore
2
Messages
1


D
Messages
Chats
Chats
Complaints
Balkans Computers
Hi, we are interested in you for...
Cyber Lab
We will upload it tomorrow
Deep & Deeper
✓ I don't agree about...
Fabulous Garden
See you tomorrow at 10 AM...
1
Cyber Lab
Project C3
Profile
A*
Internship
E*
18/04
20:11
Handshake
Hello, I am interested in this project ✓
Step 1
Handshake Accepted
Hi Jake, we look forward to know more about you!
19/04
13:11
Step 2
20/04
10:27
Hi, when will the first questionnaire be available? ✓
We will upload it tomorrow
20/04
12:55
Type here...
A
B
C
Profile
My Internships
Explore
2
Messages
1


F
By Student
By Company
B*
[19205] Jake River
Projects: F2, A1, B4, C3...
[23057] Michael Sissoko
Projects: J1
[37013] Ana Muñoz
Projects: F2, A2
[49573] Elena Borghi
Projects: I2
More...
Arise Technology
Projects: A1, A2, A3, A4...
Horizon Cloud
Projects: H1, H2, H3, H4...
Fabulous Garden
Projects: F1, F2, F3, F4...
Jet Pixel
Projects: J1, J2, J3, J4...
More...
G
My Internships
Complaints
1


G
Complaints
By Student
By Student
By Company
Michael Sissoko
Hi, I need help with an issue...
Elena Borghi
Hello, I'd like to report...
Jake River
Good day, I'm experiencing...
Ana Muñoz
Hello, I am reaching out to...
1
Elena Borghi
Projects: I2
Profile
A*
Internship
E*
Project I2
Hello, I'd like to report a problem with my ongoing internship at Infinity Code. The tasks I've been assigned differ from the initial agreement. I kindly request that this matter be addressed as soon as possible. Thank you.
21/04
17:44
F
My Internships
Complaints
1

4. Requirements Traceability

Requirements	R1: The platform allows university students to create an account. R2 : The platform allows companies to create an account. R3 : The platform allows universities to create an account. R4: The platform is able to verify the reliability of its registered users. R5: The platform allows registered users to log in.
Components	Web App API Organization Registration Service Authentication Service Identity Provider Organization Registration Service DBMS

Requirements	R6: The platform allows users to update and modify information in their profile. R7: The platform allows students to upload and publish their CVs. R9: The platform allows companies to publish available project offers along with all related details and information.
Components	Web App API User Management Service DBMS

Requirements	R8: The platform offers suggestions to students to enhance the appeal of their CV. R10: The platform offers suggestions to companies to refine the presentation of their projects. R11: The platform allows students to proactively look for internships and provides filters to help them refine their searches. R12: The platform analyses the submissions and computes recommendations that presents to both parties. R13: The system notifies students when a new project that could interest them gets published on the platform by a company. R14: The system notifies companies when a new CV that could interest them gets published on the platform by a student. R15: The platform allows students to visualize companies' profiles. R16: The platform allows companies to visualize students' profiles.

	R17: The platform allows students to search for job listings based on their skills, interests, and career goals. R18: The platform allows companies to discover potential candidates whose profiles align with job requirements, skills, and qualifications for each project. R25: The platform allows universities to visualize companies' and students' profiles.
Components	Web App API Document Analyzer Service Recommender System Explore Service Maps Notification Service Email Service Provider Maps DBMS

Requirements	R19: The platform allows companies to submit questionnaires to students they are interested in and organize interviews with them. R20: The platform allows companies to offer contracts to students they wish to engage in one of their projects. R21: The platform enables direct communication between students and companies through a messaging system. R22: The platform allows companies and students to withdraw from the selection process and internship phase at any time. R23: The platform provides students and companies with tools for tracking and monitoring the matchmaking processes and the internships they are involved in. R24: The platform collects feedback and suggestions from both students and companies and uses this information to feed the analysis mechanisms used for the recommendations.
Components	Web App API Internship Management Service Notification Service Email Service Provider Messaging Service DBMS

Requirements	R26: The platform equips universities with tools to monitor matchmaking processes and the internships in which their enrolled students participate. R27: The platform provides dedicated spaces where universities can view any complaints from both companies and related students about ongoing internships.
Components	WebApp API Internship Management Service Notification Service Email Service Provider DBMS

5. Implementation, Integration & Test Plan

5.1 Overview

This chapter explains how the described platform will be implemented and tested. The primary goal of testing is to identify and resolve most of the bugs in the code developed by the team. A detailed description of the integration process of the system's components will be provided in section 5.3. In contrast, section 5.2 will focus on the key implementation strategies employed in the project.

5.2 Implementation Plan

The system will be implemented using a combination of the bottom-up and thread strategies to leverage the advantages of both approaches. The thread strategy enables the delivery of intermediate outputs that can be evaluated by stakeholders, which is invaluable for validating the developed system. Meanwhile, the bottom-up approach supports incremental integration, facilitating effective bug tracking by allowing testing of intermediate results—subsystems formed by the integration of components—step by step as additional modules are added.

The thread strategy involves identifying the system's features and the specific portions of components, referred to as subcomponents (even if they do not strictly align with design subcomponents), responsible for delivering those features. Since the realization of a single function often requires cooperation between multiple subcomponents, defining an implementation order becomes essential. This is where the bottom-up approach proves beneficial.

This combined implementation strategy enables different development teams to work in parallel on various features. However, it is crucial to first identify shared components to prevent redundant development of the same component or subcomponent.

5.2.1 Features identification

The features to be implemented are based on the requirements outlined earlier. Some of these requirements necessitate the creation of new components, while others only require minor adjustments to existing ones. Below is a brief summary of the most important features to be implemented.

[F1] Organization Registration

This feature allows organizations (universities and companies) to register by verifying their identity by sending to the platform a set of legal documents. Identity validation is handled externally by the S&C Administrators, who can decide whether to accept or refuse their request.

[F2] Sign Up & Sign In

These features are two basic functions that can be implemented in a common way. It is important to make distinction between students, companies and universities but the interface and the interaction is similar for all parts. Please note that "Sign Up" actually only refers to students, since the organization registration procedure is different (explained in [F1]).

[F3] Creating & Refining Profile Content

This feature allows users to create and update their profiles with personal details. Companies can also create and manage projects, adding relevant details to showcase their work. Both companies and students receive suggestions about their profiles to enhance them.

[F4] Exchanging Messages

This feature enables users to send messages to others. Communication is enabled exclusively between students and companies, whether in the context of an initiated internship, a selection process (e.g., to negotiate a contract), or to request preliminary information. This feature facilitates direct interaction between students and companies within the platform.

[F5] Managing Internships

This feature facilitates the internship process based on user roles. Companies conduct interviews, make contract proposals, and manage ongoing internships, while students participate in interviews and accept or reject offers. Progress through internships is tracked by both companies and students, ensuring a collaborative and transparent process.

[F6] Job Searching

This feature enables users to search for opportunities through the global search, with the option to sort by relevance as computed by the recommender system. Students can discover projects that match their profiles, while companies can find suitable students for specific projects. Connections can be established directly through this functionality, fostering collaboration and engagement.

[F7] Collecting Feedback

This feature allows students to provide feedback to the university through a Satisfaction Survey. It integrates with the existing system components, ensuring that feedback is collected and processed without requiring any new components to be added. The functionality will be tested to ensure proper interaction with the involved components.

[F8] Complaints handling

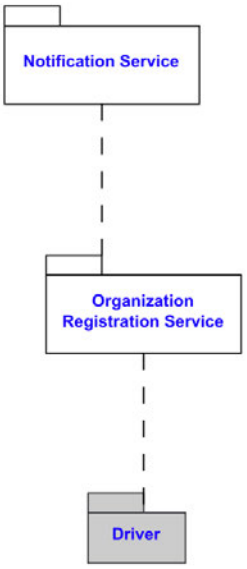
This feature enables universities to receive complaints submitted by their students or by companies. It allows universities to track and review complaints related to internships, ensuring that issues are addressed and resolved in a timely manner. Any issue management required by the complaint is handled externally, outside the context of the platform.

5.3 Component Integration and Testing

In this section, we will present the testing plan, which will involve the incremental addition of new features, requiring the testing of additional components or further portions of components already included. Please note that the DBMS is not represented, as it is assumed that, at the time of testing each feature, the DBMS and the DB are already functional for that purpose. Moreover, external components will not be represented, as they are not part of our system and are assumed to work independently.

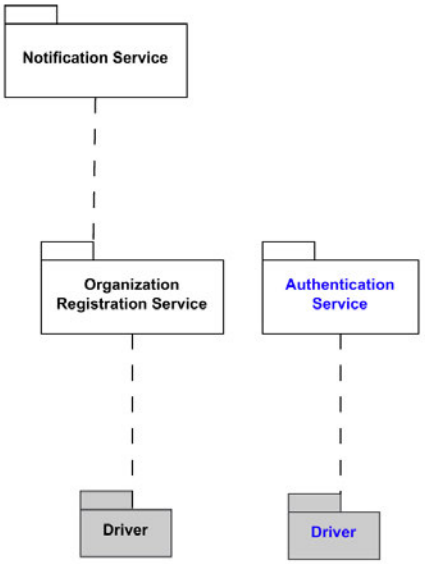
[F1] Organization Registration

The first feature we need to test is the registration of organizations. This feature involves not only the Organization Registration Service but also the Notification Service. The Email Service provider is an external component, so we can assume it functions properly.



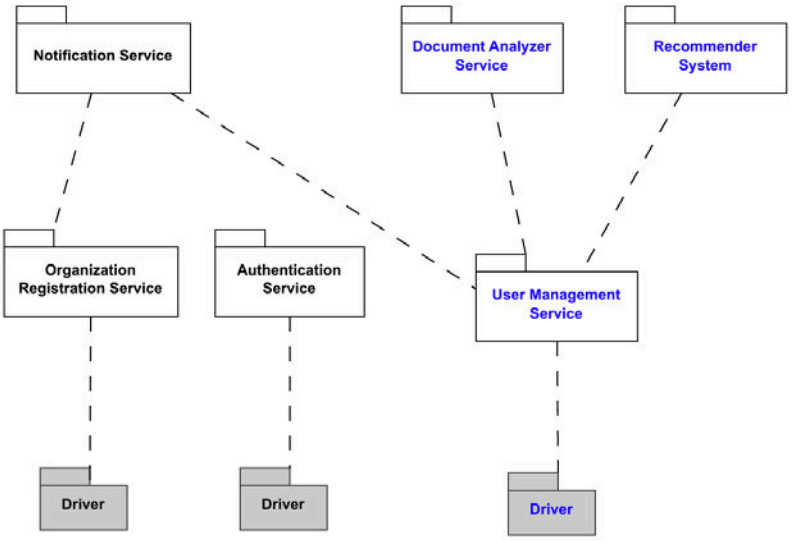
[F2] Sign Up & Sign In

At this point, we can test the student registration and login features. We can therefore include the Authentication Service. This feature also involves the Identity Provider, which again, being an external component, is assumed to be functional.



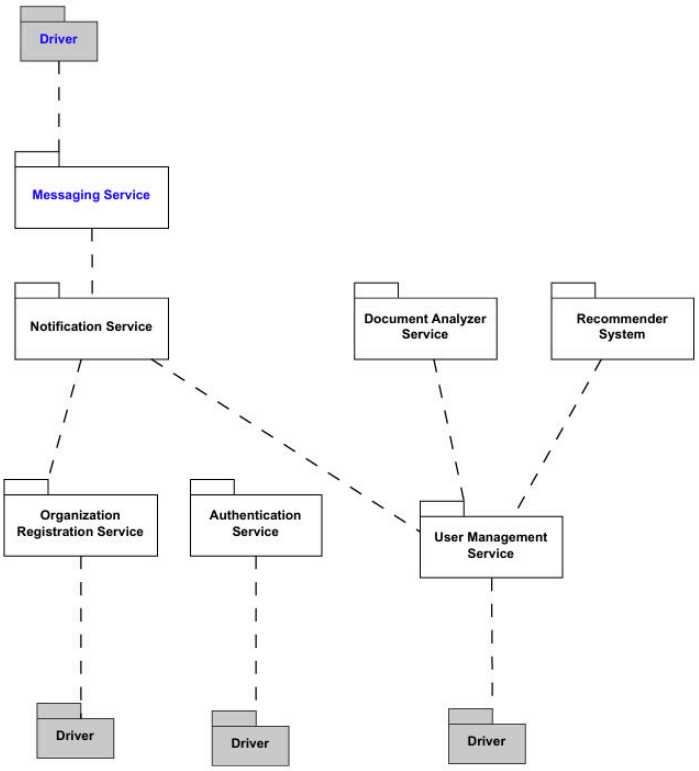
[F3] Creating & Refining Profile Content

Next, we can test the user management features, which include, for instance, adding CVs and projects. In addition to the User Management Service, we will also begin testing the Document Analyzer and the Recommender System, as well as the portion of the Notification Service related to this functionality. As in previous cases, the external Maps component is assumed to be functional.



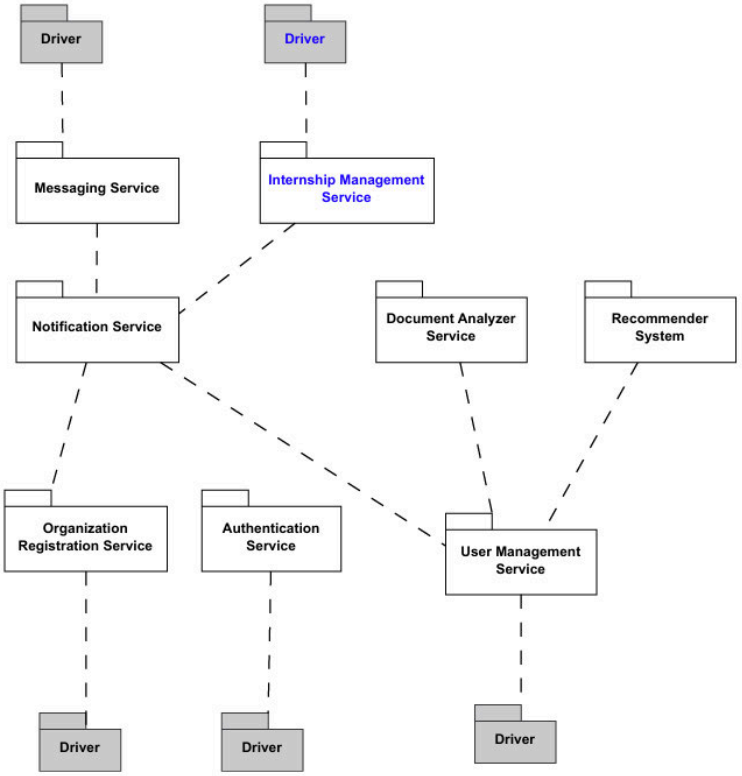
[F4] Exchanging Messages

By adding the Messaging Service, we can also test the messaging exchange functionality, which will involve the Notification Service as well.



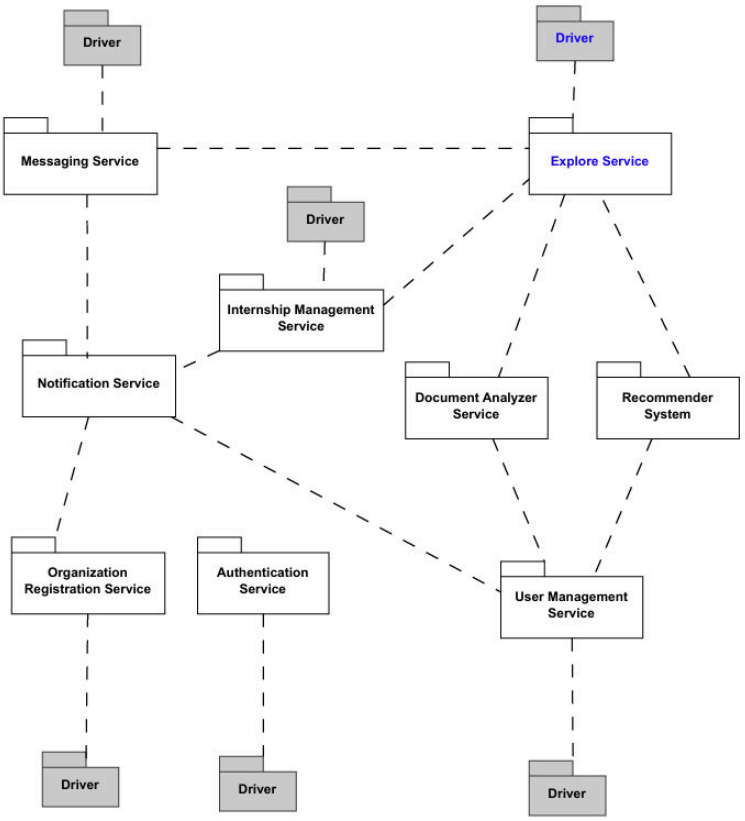
[F5] Managing Internships

The testing of the Internship Management functionality requires the addition of the Internship Management Service, and we will also test the functionality of notifications in this context.



[F6] Job Searching

We are now ready to test the Job Searching functionality, which requires adding the Explore Service to the components being tested. Additionally, it will involve testing the various portions of several components already present in the diagram that contribute to the functionality of this feature.



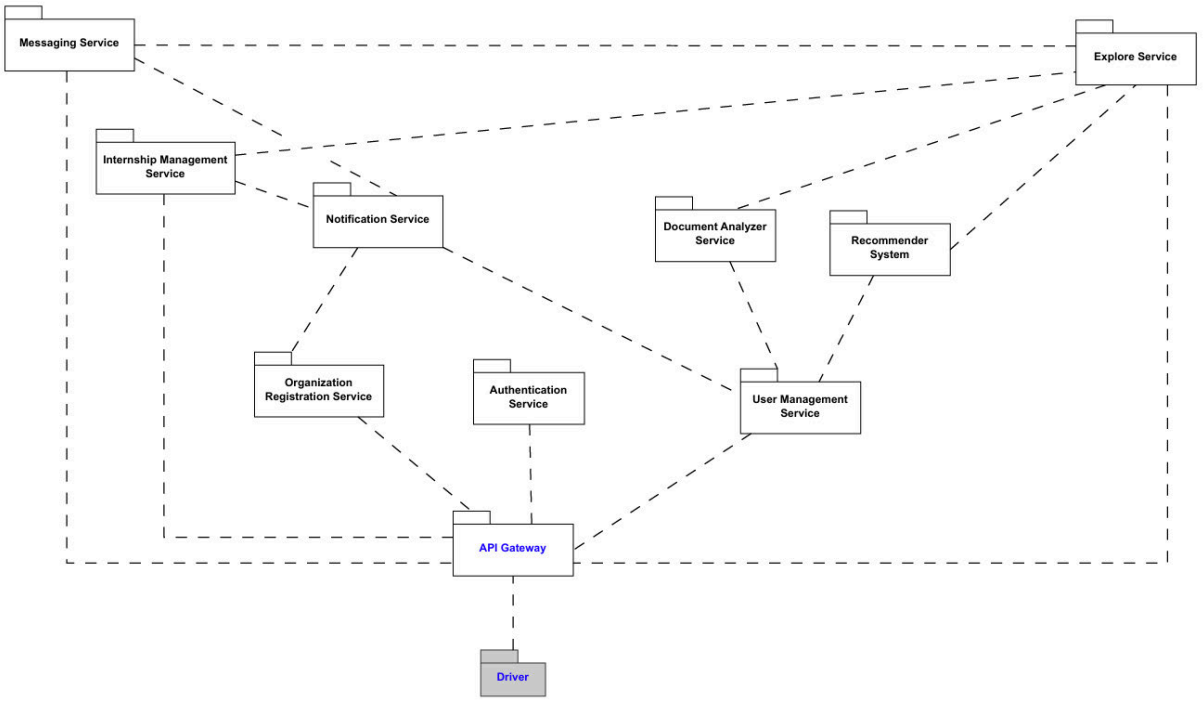
[F7] Collecting Feedback

We have already added all the functional components that are part of the system. The functionality of Collecting Feedback through the Satisfaction Survey will be tested on the involved components, but the diagram remains unchanged as no new components need to be added.

[F8] Complaints handling

The same applies to the functionality of Collecting Complaints, along with monitoring internships by universities. We will test the components needed to implement this functionality.

To conclude, we can connect the API Gateway to test the functionality of the system's complete interface.



5.4 System Testing

As previously explained in this section, developers should implement bottom-up and thread strategies for system integration and testing. These strategies are highly effective for implementing a distributed application composed of multiple services that interact with each other in various ways, as well as with APIs from many external services. During system development, each new component must be individually tested using specific scaffolders and drivers that simulate API calls from different modules. As soon as a new component is built, it is immediately integrated with the rest of the system, and available features are tested across various components. Testing integrated components is crucial, as tests need to trigger different parts of the system to ensure interactions between components work as expected. Certain functionalities, such as the Explore and Recommendation features, handle large amounts of data and numerous events and can only be fully tested with exhaustive use through prolonged system activity. Once all functionalities are verified, the system can undergo final validation and verification procedures.

Functional testing ensures that all functional and non-functional requirements outlined in the Requirement Analysis and Specification Document are met.

Performance testing analyzes the system in detail regarding non-functional requirements, particularly highlighting inefficiencies due to poor implementation that might impact system performance.

Usability testing assesses the system's accessibility, user interaction, and ease of use, which is crucial for a system used by a large user base like the S&C platform.

Load testing checks how the system behaves under specific loads or volumes of concurrent users, ensuring it can handle expected traffic and perform efficiently without crashing, slowing down, or experiencing other issues.

Finally, stress testing evaluates the system's behavior under extreme conditions or beyond normal operational capacity, identifying potential breaking points and checking how well it handles high stress.

5.5 Additional Specifications on Testing

During the entire implementation period of the system, user acceptance tests should be conducted to ensure the application meets the needs and requirements of stakeholders and end-users. Developers should implement alpha and beta testing phases. During the alpha testing phase, the application will be tested internally by the development team and selected stakeholders to identify and fix critical issues. Following this, a beta testing phase will be conducted, where a larger pool of users will use the application in a real-world environment. Security testing must be considered as a priority due to the numerous risks the system may encounter, making it essential to identify vulnerabilities as early as possible. For future releases of new features, a phased release strategy can be adopted. Initially, tested features will be made available to a small percentage of users before being expanded to the entire user base. This approach allows the developers to monitor the performance and impact of new features in a controlled environment and quickly fix possible issues before they affect all users. All testing activities should follow industry best practices and be fully documented to track issues and resolutions.

6. Effort Spent

The following table displays the number of hours each group member spent on the various sections of the document. Please note that the division is only approximate, and each section still required the collaboration of all members.

	Chapter 1	Chapter 2	Chapter 3	Chapter 4	Chapter 5
	5	8	1	12	5
	1	14	8	1	8
	2	16	1	3	9

7. References

Documentation about SSO authentication and Identity Provider :
https://learn.microsoft.com/en-us/entra/identity/authentication/
Sequence diagrams are made with Astah UML.
Architecture diagrams are made with draw.io.
UI mockups are made with Figma.